

Tugas Besar 2 IF3070 Dasar Inteligensi Artifisial

Implementasi Algoritma Machine Learning



DISUSUN OLEH TIM RestInPeaceMom:

Muhammad Aymar Barkhaya	18223051
Surya Suharna	18223075
Ni Made Sekar Jelita Parameswari	18223101
Muhammad Azzam Robbani	18223025

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
FAKULTAS SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2025

Daftar Isi

Daftar Isi	2
Daftar Tabel	4
Deskripsi Persoalan	5
Pembahasan	6
1. Implementasi Decision Tree Learning (DTL)	6
2. Implementasi Logistic Regression	7
3. Implementasi KNN	9
4. Exploratory Data Analysis	9
4.1 Inspecting Data	9
4.2 Exploring Data Characteristics	11
4.3 Visualizing Relationship of Data	12
5. Data Splitting	18
5.1 Strategi Pembagian Data	19
5.2 Penerapan Stratified Sampling	19
6. Data Cleaning	19
6.1 Handling Missing Data	19
6.2 Dealing with Outliers	19
6.3 Data Validation	20
6.4 Remove Duplicates	21
6.5 Feature Engineering	21
6.6 Pipeline Data Cleaning	24
7. Data Preprocessing	25
7.1 Feature Scaling	25
7.2 Dimensionality Reduction	26
7.3 Handling Imbalanced Data	27
7.4 Pipeline Data Preprocessing	28
8. Perbandingan Hasil Prediksi dengan Pustaka	29
Bonus	33
9. Gambar Percabangan Tree (DTL)	33
10. Garis Kontur Fungsi Loss (log-loss)	37
11. Proses Training Model KNN	38

12. Optimasi Model	39
12.1 Hyperparameter Tuning DTL	39
12.2 Hyperparameter Tuning Logistic Regression	40
Kesimpulan dan Saran	44
13. Kesimpulan	44
14. Saran	44
Specification Traceability	46
Tabel 7.1 Tabel Specification Traceability	46
Pembagian Tugas	48
Tabel 8.1 Pembagian Tugas Anggota Kelompok	48
Referensi	49

Daftar Tabel

Tabel 7.1 Tabel Specification Traceability.....	46
Tabel 8.1 Pembagian Tugas Anggota Kelompok.....	48

Deskripsi Persoalan

Tugas besar ini bertujuan memberikan pengalaman langsung menerapkan algoritma pembelajaran mesin pada dataset nyata dan kompleks. Persoalan utamanya adalah melakukan klasifikasi biner memprediksi variabel target pada kumpulan data yang besar (sekitar 100 ribu baris). Dataset ini secara sengaja dirancang menyerupai pola data fraud dunia nyata, mencakup nilai yang hilang (missing values), outlier, dan interaksi fitur yang rumit, sehingga menuntut serangkaian tahapan pra-pemrosesan data yang cermat. Tahapan tersebut mencakup Data Cleaning untuk menangani nilai yang hilang dan data tidak valid, Data Transformation (seperti encoding variabel kategorikal dan scaling fitur numerik), hingga Feature Selection atau Dimensionality Reduction.

Tantangan inti dari tugas ini adalah implementasi dua algoritma pembelajaran mesin secara mandiri (from scratch). Algoritma yang wajib diimplementasikan adalah Decision Tree Learning (DTL), memilih antara C4.5 atau CART dengan persyaratan minimal mampu menangani tipe data numerik dan kategorikal, serta mengatasi null values. Algoritma kedua harus dipilih antara Logistic Regression atau K-Nearest Neighbors (KNN), juga diimplementasikan from scratch. Semua implementasi mandiri ini harus diorganisir dalam bentuk kelas-kelas (class DTL, class KNN, dsb.) dan hanya diperbolehkan menggunakan pustaka eksternal untuk perhitungan matematika dasar seperti NumPy.

Setelah implementasi from scratch selesai, kinerja model-model tersebut wajib divalidasi dan dibandingkan dengan hasil yang diperoleh dari implementasi yang setara menggunakan pustaka standar scikit-learn. Perbandingan ini harus didokumentasikan secara rinci dalam laporan, mencakup perbandingan metrik yang sesuai. Selain itu, tugas ini mencakup komponen wajib seperti penyimpanan dan pemuatan model, serta Kaggle Submission minimal satu kali untuk menguji kinerja pada leaderboard. Terdapat juga poin bonus yang dapat diperoleh melalui peringkat tinggi di Kaggle dan implementasi algoritma optimasi model tambahan yang berada di luar materi kelas.

Pembahasan

1. Implementasi Decision Tree Learning (DTL)

Decision Tree Learning adalah algoritma Supervised Learning yang bisa digunakan untuk persoalan klasifikasi maupun regresi dalam Machine Learning. Algoritma ini akan membuat model yang dapat memprediksi nilai suatu variabel target berdasarkan *decision rules* yang diperoleh dari variabel-variabel fitur training dataset. Struktur pohon ini terdiri atas node-node yang merepresentasikan hasil keputusan dan cabang-cabang yang merepresentasikan konsekuensi keputusan tersebut.

Pada implementasi kami, struktur tree dibangun di atas dua kelas fundamental, yaitu Node dan DecisionTreeClassifier. Kelas Node berperan sebagai decision node yang menyimpan logika split maupun sebagai leaf node yang menyimpan nilai prediksi akhir. Atributnya adalah indeks fitur split, threshold split, dan information gain. Sementara itu, kelas DecisionTreeClassifier berperan sebagai pengelola utama yang menginisialisasi hyperparameter. Hyperparameter yang digunakan adalah `min_samples_split` (jumlah minimal data untuk membelah node) dan `max_depth` (kedalaman maksimal pohon), serta menyimpan referensi ke akar pohon (root) setelah struktur terbentuk.

Proses pelatihan model dimulai dari fungsi `fit` yang memanggil metode rekursif `_build_tree` untuk menyusun struktur pohon secara bertahap dari data train. Di setiap rekursi, algoritma mencari pembagian data terbaik menggunakan fungsi `_get_best_split` dengan cara mengiterasi setiap fitur. Jika variasi nilai fitur sangat tinggi (lebih dari 100 nilai unik), algoritma menggunakan pendekatan persentil untuk menentukan kandidat threshold demi efisiensi. Rekursi ini akan berhenti jika memenuhi stopping criteria, yaitu ketika kedalaman maksimum tercapai, jumlah sampel kurang dari batas minimum, atau node sudah menjadi murni.

Untuk menentukan kualitas setiap kemungkinan split, implementasi ini menggunakan perhitungan Information Gain yang didasarkan pada metrik Gini Impurity. Fungsi `_gini` menghitung tingkat kemurnian data pada sebuah node, di mana nilai 0 menandakan node tersebut murni alias hanya terdapat 1 kategori sampel. Dalam fungsi `_information_gain`, algoritma menghitung seberapa besar Gini Impurity berkurang setelah data di-split, rumusnya selisih antara Gini awal dengan rata-rata weighted Gini dari sub node kiri dan kanan. Inilah yang menjadi skor gain, di mana skor tertinggi akan dipilih sebagai decision rule terbaik.

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

Setelah pohon terbentuk, klasifikasi data baru dilakukan melalui fungsi predict yang memproses input menggunakan metode bantuan `_make_prediction`. Metode ini bekerja dengan menelusuri pohon dari akar hingga ke ujung bawah. Pada setiap branch, nilai fitur data dibandingkan dengan threshold yang tersimpan. Jika nilai fitur lebih kecil atau sama dengan threshold, alur akan berbelok ke cabang kiri, dan sebaliknya ke kanan, hingga akhirnya mencapai leaf node yang mengembalikan nilai label mayoritas sebagai hasil prediksi menggunakan fungsi `calculate_leaf_value`.

2. Implementasi Logistic Regression

Logistic Regression merupakan algoritma klasifikasi yang bertujuan memprediksi probabilitas suatu data masuk ke dalam salah satu dari dua kelas biner. Dalam implementasi ini, model diinisialisasi dengan parameter utama berupa **learning rate** untuk mengatur kecepatan belajar, **jumlah iterasi** untuk menentukan durasi pelatihan, dan **class weight** yang berfungsi menangani ketidakseimbangan data. Saat proses pelatihan dimulai melalui fungsi fit, bobot (weights) dan bias model diatur ke nilai nol sebagai titik awal sebelum dilakukan optimasi.

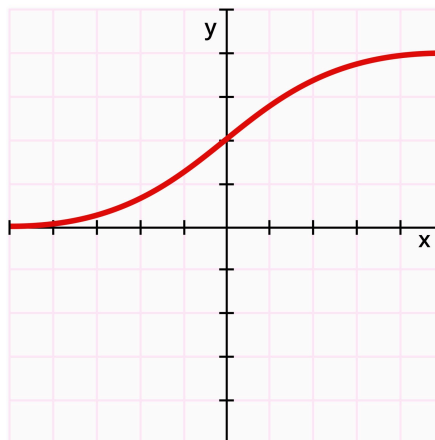
Tahap pembelajaran dimulai dengan mekanisme yang disebut *forward pass*, yaitu alur perhitungan satu arah untuk mengubah data mentah menjadi angka prediksi. Proses ini diawali dengan perhitungan skor linier (raw score), di mana model mengalikan data input dengan bobot (weights) yang sedang dipelajari, lalu menambahkannya dengan bias. Hasil dari operasi matematika ini adalah sebuah nilai angka bebas yang belum memiliki batas; nilainya bisa melonjak sangat tinggi (positif) atau jatuh sangat rendah (negatif) tergantung pada karakteristik datanya.

Sebelum skor mentah tersebut diolah lebih lanjut, kode ini menerapkan langkah teknis yang sangat penting bernama mekanisme clipping atau penstabil nilai. Pada tahap ini, skor secara otomatis "dijepit" agar tetap berada dalam rentang aman, yaitu

antara -500 hingga 500. Langkah ini dilakukan untuk mengatasi keterbatasan komputer dalam menghitung angka eksponensial. Tanpa pembatasan ini, skor yang terlalu ekstrem (misalnya 1.000) akan menyebabkan numerical overflow yakni kondisi di mana hasil perhitungan melampaui kapasitas memori yang menghasilkan nilai NaN (Not a Number) dan merusak seluruh proses pelatihan.

Evaluasi performa model dilakukan menggunakan fungsi Weighted Binary Cross-Entropy Loss. Berbeda dengan perhitungan loss standar, fungsi ini memperhitungkan bobot kelas (class weight) sehingga kesalahan prediksi pada kelas minoritas (seperti data fraud) akan diberikan hukuman penalti yang lebih besar daripada kelas mayoritas. Nilai loss inilah yang menjadi dasar proses Gradient Descent. Pada tahap ini, gradien dihitung dengan mengalikan selisih error prediksi dengan bobot sampel masing-masing, lalu parameter bobot dan bias diperbarui secara berulang (iteratif) bergerak berlawanan arah dengan gradien untuk meminimalkan kesalahan hingga model mencapai performa optimal.

Logistic Function



Fungsi ini bertugas memampatkan angka tadi menjadi nilai desimal yang rapi dalam rentang 0 hingga 1. Nilai akhir inilah yang keluar sebagai prediksi probabilitas (misalnya, angka 0.85 menunjukkan peluang 85% bahwa data tersebut adalah fraud), yang selanjutnya siap dibandingkan dengan label asli untuk menghitung tingkat kesalahan model.

3. Implementasi KNN

Pada modul ini, kami mengembangkan classifier K-Nearest Neighbors (KNN) yang bekerja berdasarkan prinsip lazy learning. Berbeda dengan algoritma yang melakukan pelatihan berat di awal, pendekatan ini menjadikan fase fit sangat ringan; sistem hanya bertugas memuat dan menyimpan data latih (X_{train} dan y_{train}) ke dalam memori sebagai referensi. Beban komputasi yang sebenarnya baru akan terjadi sepenuhnya pada tahap prediksi, di mana algoritma harus membandingkan kedekatan antara data baru dengan seluruh basis data yang tersimpan.

Tantangan utama dalam menangani dataset besar adalah efisiensi waktu dan memori. Untuk mengatasinya, kami meninggalkan metode looping konvensional yang lambat dan beralih ke strategi vektorisasi pada fungsi `_calculate_distances_vectorized`. Khusus untuk perhitungan jarak Euclidean, kami menerapkan manipulasi aljabar dengan rumus $(a - b)^2 = a^2 - 2ab + b^2$. Teknik ini memungkinkan perhitungan kuadrat matriks dilakukan secara terpisah dan disatukan kembali melalui operasi dot product, yang secara signifikan mempercepat proses dibandingkan metode pengurangan elemen per elemen.

Sementara itu, untuk metrik jarak Manhattan, kami menerapkan penanganan khusus guna menghindari *MemoryError*. Karena perhitungan selisih absolut antar matriks raksasa membutuhkan alokasi memori yang masif, kami mengadopsi teknik chunking. Proses komputasi dipecah menjadi blok-blok kecil; misalnya, memproses sekumpulan data uji terhadap 5000 data latih secara bergantian. Strategi ini memastikan penggunaan RAM tetap terkendali tanpa mengorbankan integritas hasil perhitungan.

Optimasi lebih lanjut dilakukan pada mekanisme pencarian tetangga terdekat. Daripada mengurutkan (sorting) seluruh jarak yang memakan biaya komputasi tinggi, kami memanfaatkan fungsi `numpy.argpartition`. Metode ini jauh lebih efisien karena hanya menyeleksi k elemen terkecil tanpa perlu mengurutkan sisa data lainnya. Terakhir, seluruh proses prediksi dibungkus dalam mekanisme batch processing (default 100 sampel per batch) untuk menjaga stabilitas sistem saat melakukan inferensi pada ribuan data sekaligus.

4. Exploratory Data Analysis

4.1 Inspecting Data

Langkah awal dalam memahami karakteristik dataset fraud detection ini dimulai dengan meninjau struktur dan konten data secara menyeluruh. Proses ini krusial untuk menentukan strategi pra-pemrosesan yang tepat sebelum pemodelan dilakukan.

4.1.1 Inspecting Data: Tinjauan Dimensi dan Struktur Data

Kami memulai dengan memeriksa dimensi dataset train.csv yang terdiri dari 100.000 baris dengan 31 kolom (30 fitur dan 1 variabel target). Secara umum, tipe data terbagi menjadi dua kategori utama:

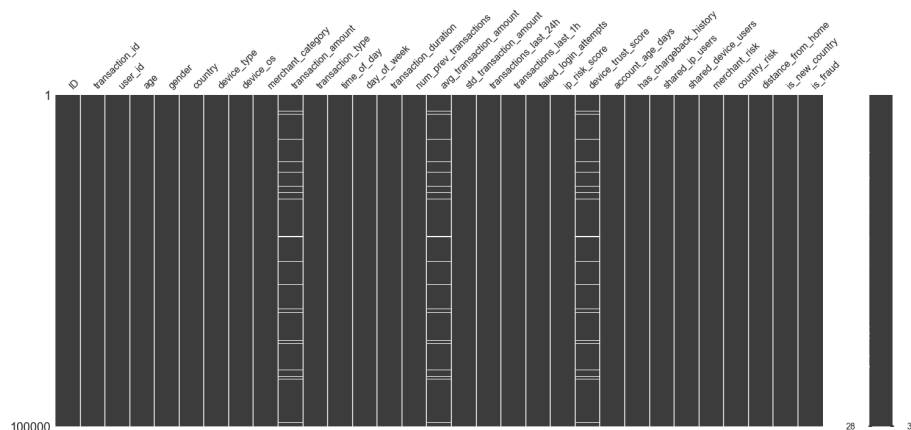
- Fitur Numerik (24 kolom): Terdiri dari tipe data float64 dan int64 yang mencakup informasi-informasi seperti nilai transaksi, skor risiko, durasi, dan lain sebagainya.
- Fitur Kategorikal (7 kolom): Bertipe object, mencakup atribut seperti jenis perangkat, sistem operasi, kategori merchant, dan sebagainya.

4.1.2 Identifikasi Missing Values

Terdapat null value pada tiga fitur numerik, yaitu sebagai berikut.

- transaction_amount: 2.419 data hilang
- avg_transaction_amount: 2.419 data hilang
- device_trust_score: 2.419 data hilang

Total terdapat 7.257 nilai yang hilang di seluruh dataset. Kami memvisualisasikan pola kekosongan data ini menggunakan matriks missingno untuk melihat apakah ada korelasi antar fitur yang hilang, yang kemudian akan ditangani pada tahap data cleaning.



4.1.3 Identifikasi Unique Values pada Categorical Features

Kami meninjau *unique values* pada data-data kategorikal (device type, device os, dan transaction type). Ditemukan bahwa tipe transaksi yang paling umum adalah *purchase*, device os yang paling banyak digunakan adalah Android, dan device type yang paling banyak digunakan adalah mobile.

```
Transaction Type Description and Unique Values:
```

```
count      100000
```

```
unique           4
```

```
top      purchase
```

```
freq      59395
```

```
Name: transaction_type, dtype: object
```

```
['purchase' 'topup' 'withdrawal' 'transfer']
```

```
Device OS Description and Unique Values:
```

```
count      100000
```

```
unique           5
```

```
top      Android
```

```
freq      49183
```

```
Name: device_os, dtype: object
```

```
['Android' 'iOS' 'Windows' 'Linux' 'MacOS']
```

```
Device Type Description and Unique Values:
```

```
count      100000
```

```
unique           3
```

```
top      mobile
```

```
freq      78292
```

```
Name: device_type, dtype: object
```

```
['mobile' 'desktop' 'tablet']
```

4.2 Exploring Data Characteristics

4.2.1 Statistik Deskriptif

Hasil statistik deskriptif yang dijalankan untuk data numerik dari dataset menunjukkan informasi-informasi penting: count, mean, std, min, Q1, Q2 (mean), Q3, dan max.

4.2.2 Statistics Summary

Kami juga menjalankan rangkuman statistik yang lebih lengkap dengan menambahkan median, mode, skewness, dan kurtosis pada data kemudian menaruhnya dalam tabel sehingga mempermudah proses perbandingan antarfitur, terutama skewness dan kurtosis.

```
# Summary Statistics of Numerical Features including Skewness and Kurtosis
```

```
summary = pd.DataFrame()
```

```
summary["min"] = train_df.min(numeric_only=True)
```

```
summary["max"] = train_df.max(numeric_only=True)
```

```
summary["mean"] = train_df.mean(numeric_only=True)
```

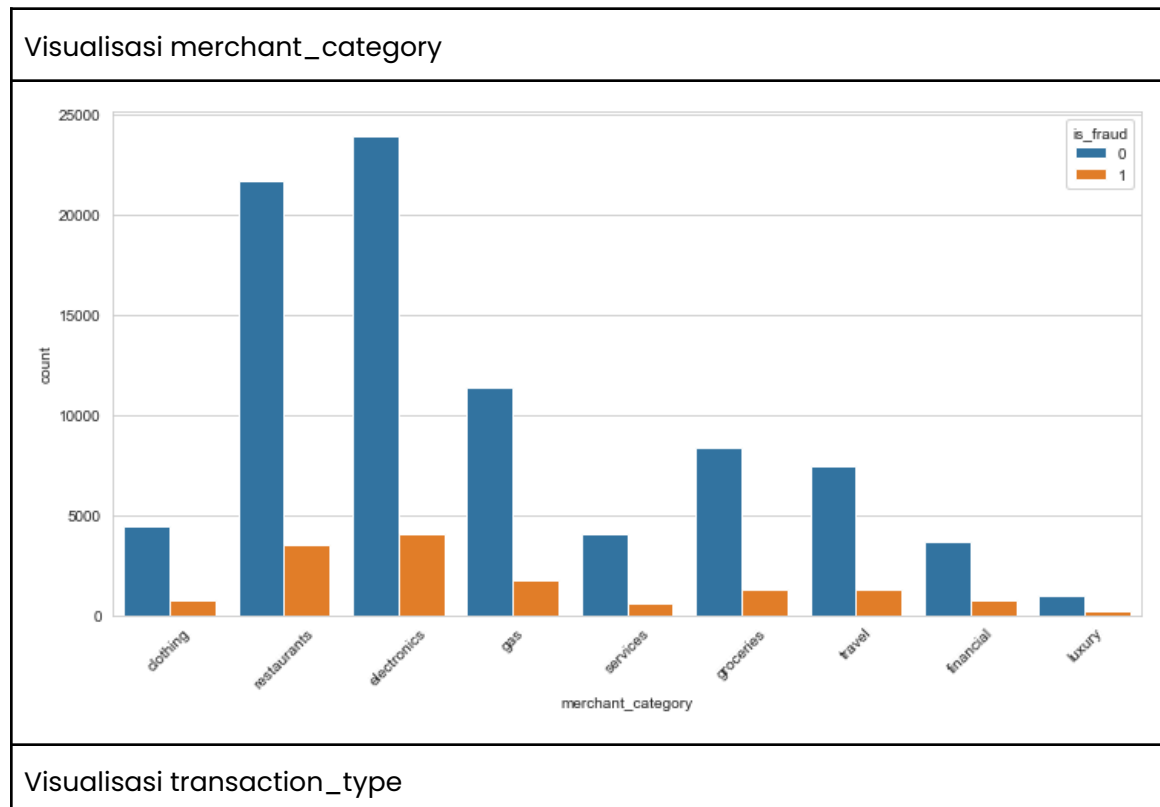
```
summary["median"] = train_df.median(numeric_only=True)
summary["mode"] = train_df.mode(numeric_only=True).iloc[0]
summary["std"] = train_df.std(numeric_only=True)
summary["skewness"] = train_df.skew(numeric_only=True)
summary["kurtosis"] = train_df.kurtosis(numeric_only=True)
```

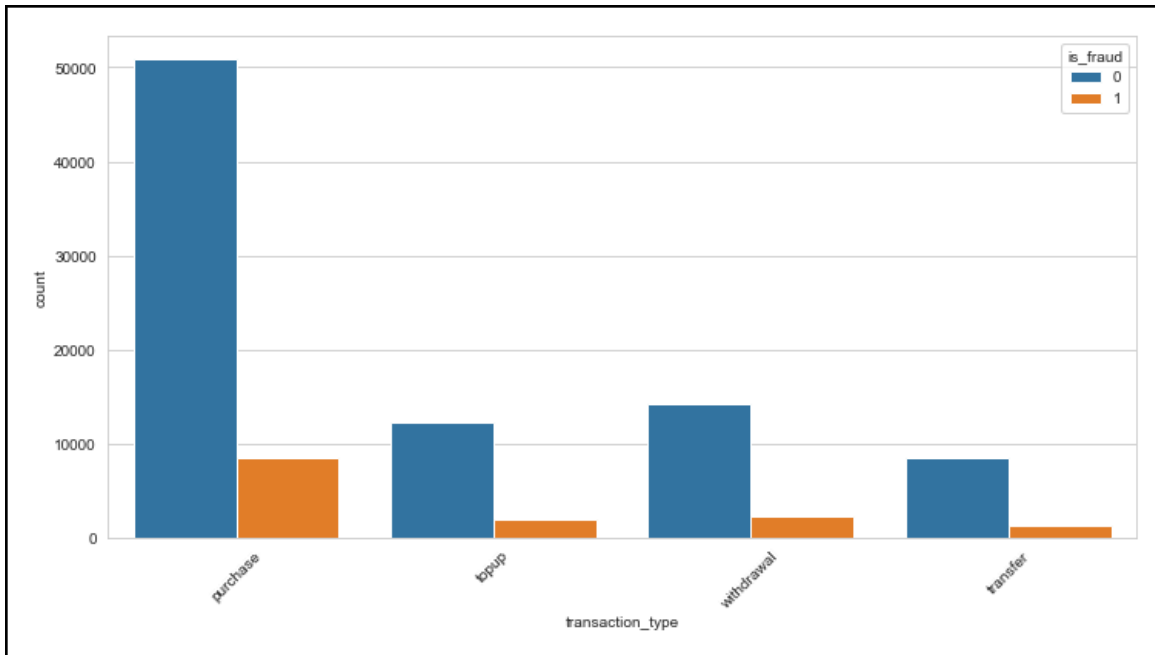
summary

4.3 Visualizing Relationship of Data

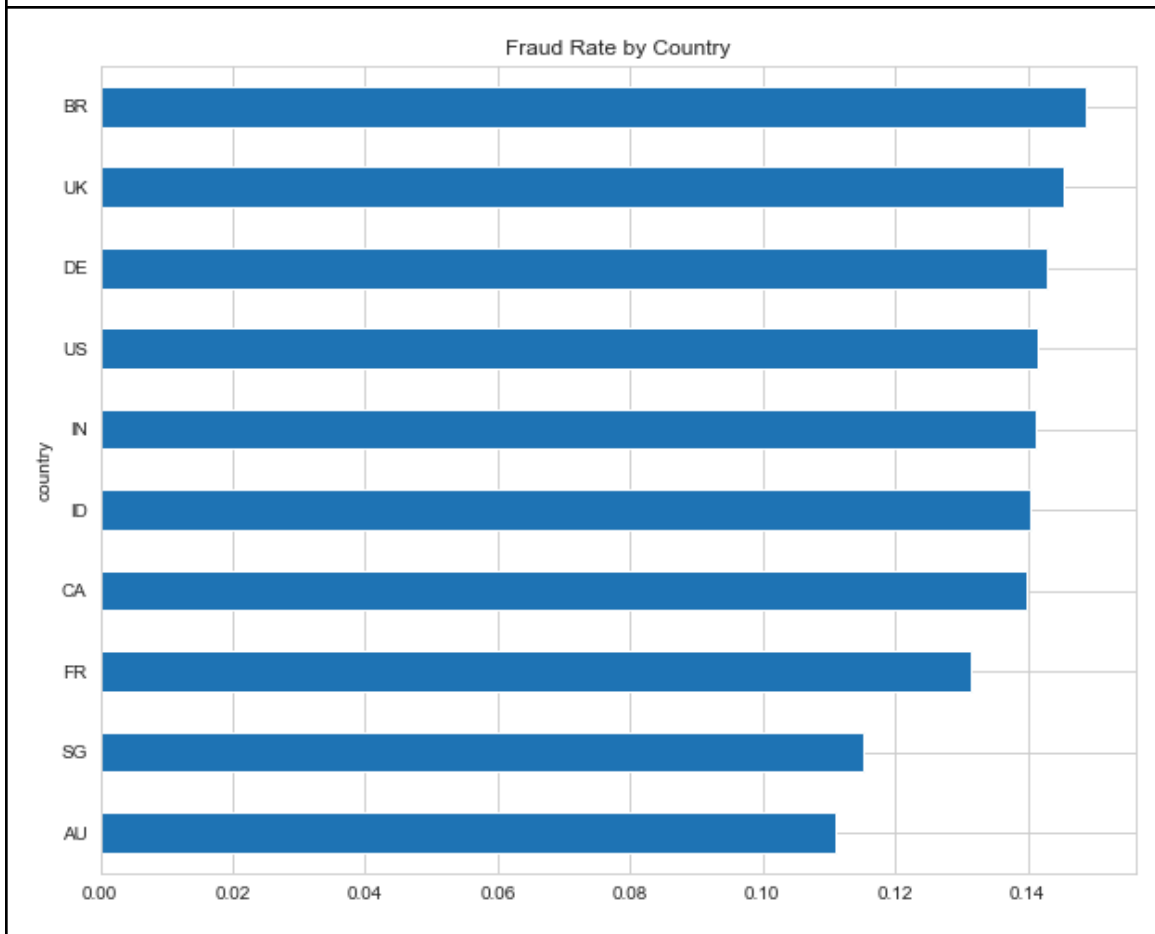
4.3.1 Categorical Data

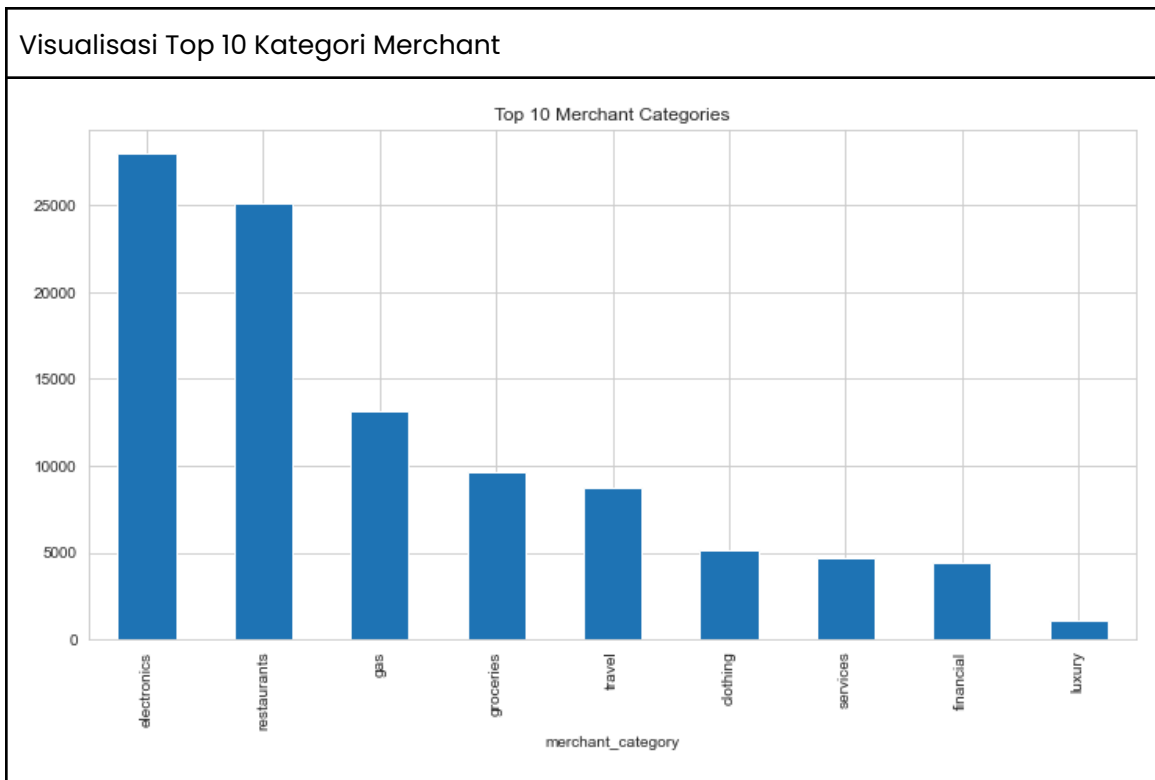
Berikut adalah visualisasi data yang kami jalankan pada data kategorikal.



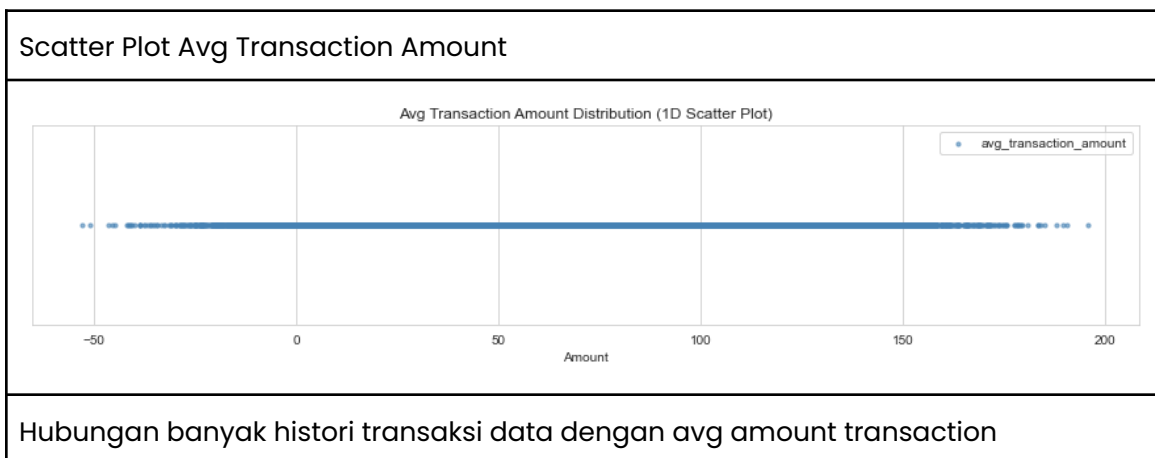


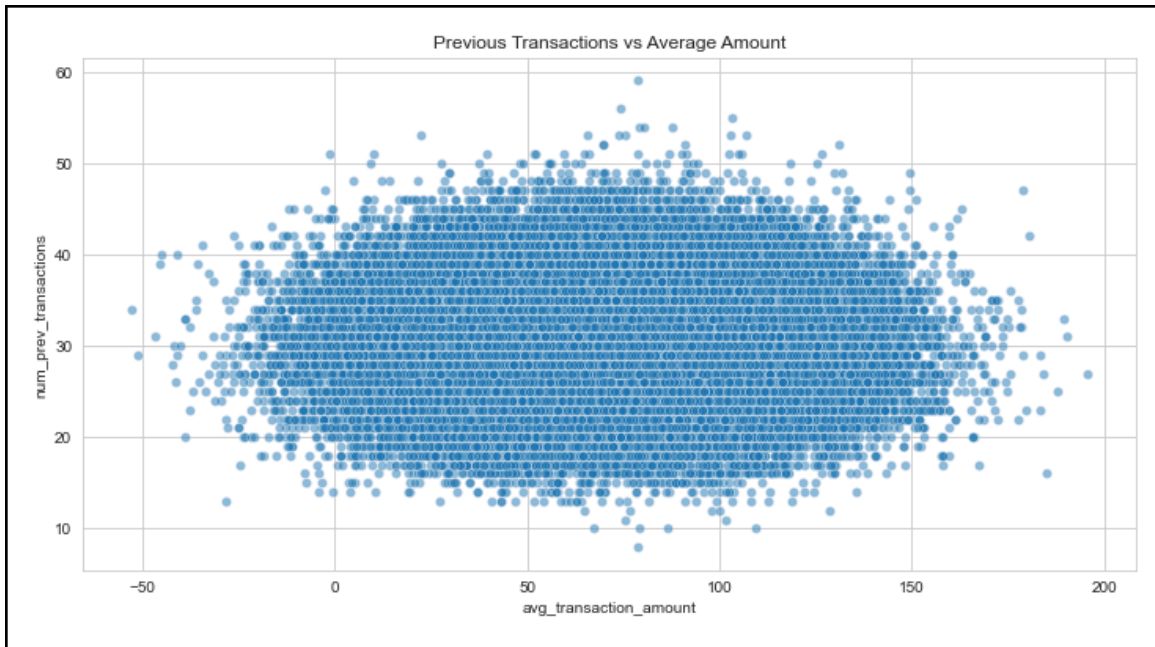
Visualisasi *Fraud Rate* berdasarkan country



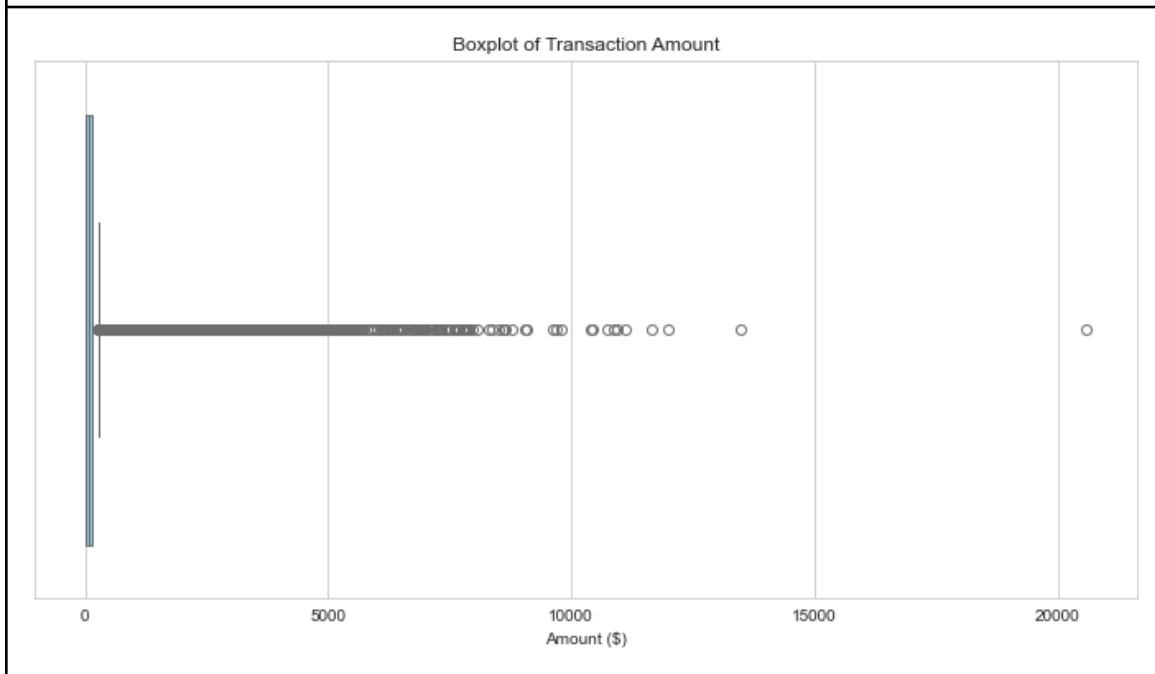


4.3.2 Numerical Data



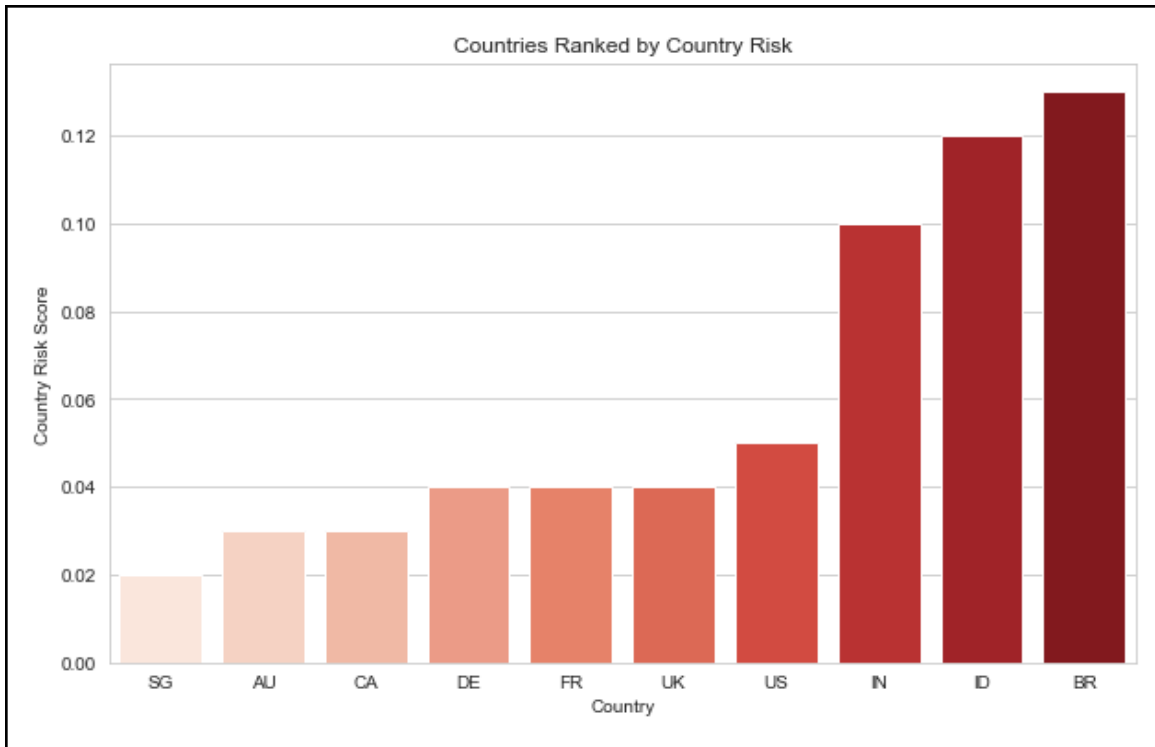


Boxplot Transaction Amount

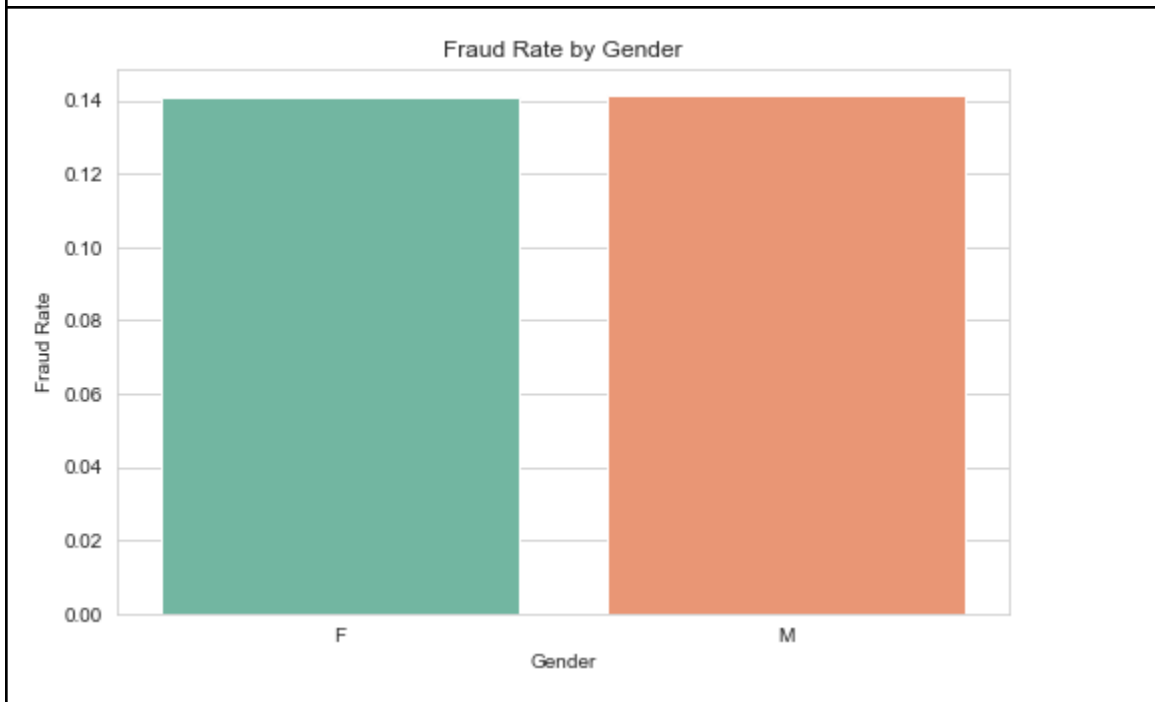


4.3.3 Relationship

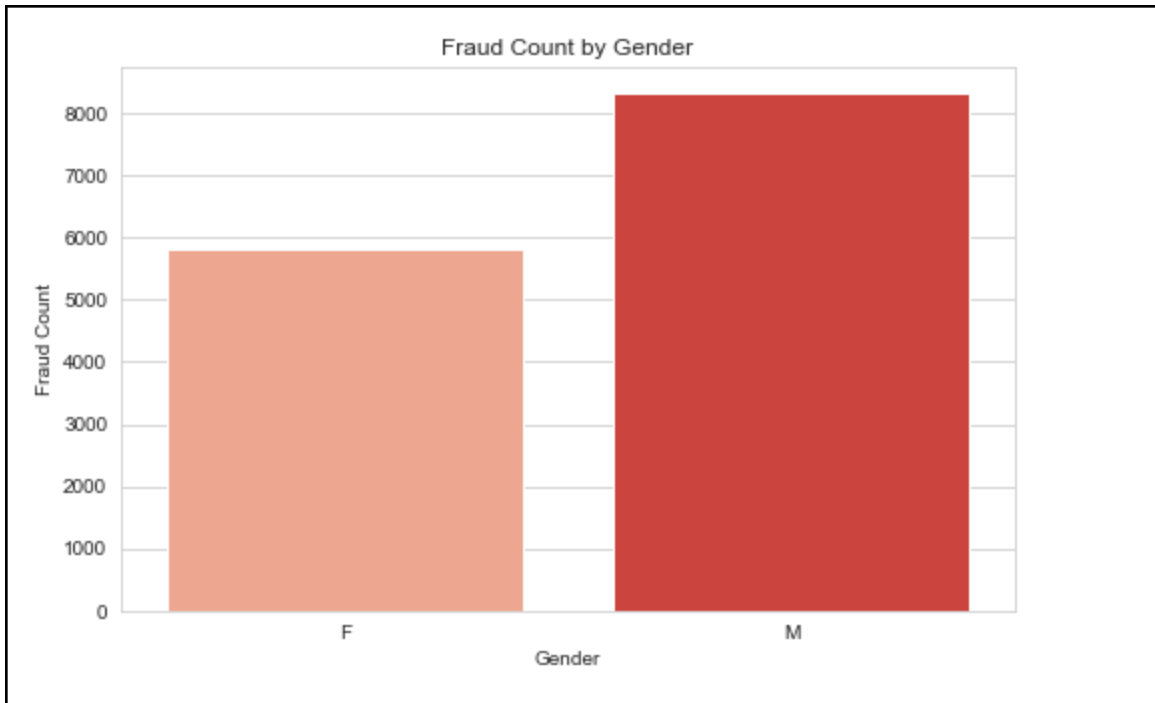
Rank negara berdasarkan Country Risk



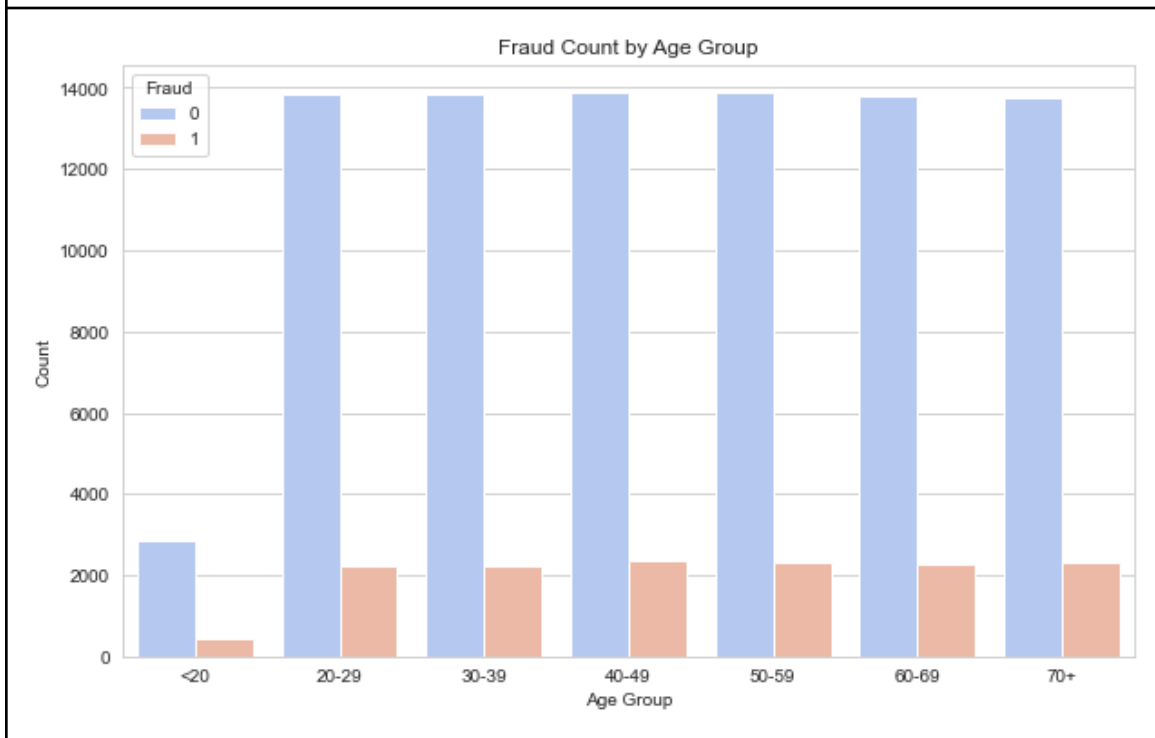
Fraud Rate berdasarkan gender



Fraud Count berdasarkan gender



Fraud Count berdasarkan kelompok usia (yang sudah di-bin)



4.2.2 Distribusi Kategori Transaksi

Melalui countplot, kami menganalisis distribusi jenis transaksi. Hasil visualisasi menunjukkan bahwa mayoritas transaksi didominasi oleh kategori

purchase (pembelian), diikuti oleh jenis transaksi lainnya. Memahami frekuensi ini penting karena tipe transaksi tertentu mungkin memiliki kerentanan lebih tinggi terhadap aktivitas fraud.

4.2.3 Tingkat Fraud Berdasarkan Negara

Kami memetakan rata-rata kejadian fraud berdasarkan negara asal pengguna menggunakan bar chart. Visualisasi ini memberikan wawasan mengenai lokasi geografis mana yang memiliki tingkat risiko kecurangan lebih tinggi, yang bisa menjadi fitur penting dalam model prediksi.

4.2.4 Distribusi Durasi dan Pola Transaksi

Histogram pada fitur `transaction_duration` menunjukkan distribusi data yang condong (skewed), yang mengonfirmasi perlunya normalisasi atau penanganan outlier di tahap selanjutnya. Selain itu, kami menggunakan scatterplot untuk melihat hubungan antara jumlah transaksi sebelumnya (`num_prev_transactions`) dengan rata-rata jumlah transaksi (`avg_transaction_amount`) guna mendeteksi pola anomali perilaku pengguna.

5. Data Splitting

Sebelum melakukan *data cleaning* dan transformasi lebih lanjut, kami membagi dataset menjadi training set dan validation set. Langkah ini dilakukan di awal untuk mencegah *data leakage*: informasi dari data uji secara tidak sengaja "bocor" ke dalam proses pelatihan model.

```
X = raw_data.drop('is_fraud', axis=1) # features
y = raw_data['is_fraud'] #target feature: is_fraud

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

# X_train 80% feature
# y_train 80% of target values
# X_val 20% feature
# y_val 20% of target values

print(f"Training set size: {len(X_train):,}")
print(f"Validation set size: {len(X_val):,}")
print(f"\nTraining set fraud rate: {y_train.mean()*100:.2f}%")
print(f"Validation set fraud rate: {y_val.mean()*100:.2f}%")
```

5.1 Strategi Pembagian Data

Kami menggunakan fungsi `train_test_split` dari pustaka Scikit-Learn dengan rasio pembagian 80:20.

- a. Training Set (80%): Berjumlah 80.000 sampel, digunakan untuk melatih model.
- b. Validation Set (20%): Berjumlah 20.000 sampel, digunakan untuk evaluasi performa model selama eksperimen.

5.2 Penerapan Stratified Sampling

Mengingat dataset ini bersifat tidak seimbang,, kami menerapkan parameter `stratify=y` saat melakukan pemisahan. Teknik ini memastikan bahwa proporsi kelas target (`is_fraud`) tetap konsisten di kedua kelompok data. Setelah dijalankan, hasil verifikasi menunjukkan bahwa baik training set maupun validation set memiliki tingkat fraud yang identik, yaitu sebesar 14,13%, sehingga model akan dilatih dan divalidasi pada distribusi data yang representatif.

6. Data Cleaning

6.1 Handling Missing Data

Null value yang sudah ditemukan sebelumnya hanya ditemukan pada tiga feature sehingga kami akan menganalisis lebih lanjut tindakan apa yang harus dikenakan pada ketiga feature tersebut. Setelah dibandingkan jumlah null value dengan yang tidak, ditemukan bahwa ketiga *feature* memiliki persentase null value yang cukup kecil yaitu 2.419%. Dengan pertimbangan tersebut, kami memutuskan untuk meng-*handle* null value dengan melakukan *imputation* dengan median value. Meskipun begitu, `transaction_amount` memiliki skewness yang cukup tinggi. Oleh karena itu, agar tidak menimbulkan ambiguitas, kami juga memasang *feature indicator* berupa value 1 atau 0 yang mengindikasikan bahwa *row* tersebut sebelumnya memiliki null value (`device_trust_score_missing`, `avg_transaction_amount_missing`, `transaction_amount_missing`).

6.2 Dealing with Outliers

Dalam kasus menangani *fraud detection*, data yang termasuk outliers memiliki kemungkinan besar merupakan data yang mengindikasikan *fraud*. Oleh karena itu, outlier tidak dihapus ataupun di-impute. Akan tetapi, seiring berjalannya pengerjaan tugas ini, kami terus bereksperimen dan mencoba menambah, menghapus outliers, dan bahkan menggunakan metode *Capping*. Didapat kesimpulan bahwa submisi di Kaggle dengan Logistic Regression

meningkat ketika data outliersnya dihapus dan sebaliknya untuk DTL. Hal ini dapat terjadi karena Logistic Regression membutuhkan outlier tersebut. Dalam deteksi *fraud*, nilai ekstrem seringkali merupakan sinyal utama kecurangan sehingga menghapus *row* tersebut sama dengan membuang data *fraud* yang ingin dideteksi. Sebaliknya, DTL bekerja dengan membagi data berdasarkan aturan-aturan sehingga keberadaan outlier yang terlalu ekstrem justru memicu overfitting (*tree* membuat cabang yang terlalu spesifik hanya untuk mengakomodasi anomali yang sedikit). Oleh karena itu, menghapus data outlier membantu DTL menghasilkan aturan yang lebih rapi.

6.3 Data Validation

Pada tahap ini, dilakukan validasi data berdasarkan penjelasan data yang diberikan oleh asisten. Validasi data-data ini mencakup mengidentifikasi *feature-feature* yang harus memiliki $value > 0$, tidak boleh melebihi *range* tertentu, dan lain sebagainya. Selengkapnya pada tabel berikut.

```
def check_value_validity(X):
    print("transaction_amount:" , (X['transaction_amount'] < 0).sum())
    print("device_trust_score:" , (X['device_trust_score'] > 1).sum())
    print("time_of_day:" , ((X['time_of_day'] < 0) | (X['time_of_day'] >
23)).sum())
    print("day_of_week:" , ((X['day_of_week'] < 0) | (X['day_of_week'] >
6)).sum())
    print("transaction_duration:" , (X['transaction_duration'] < 1).sum(),
" , " , (X['transaction_duration'] < 1).sum()/len(X) * 100, "%")
    print("avg_transaction_amount:" , (X['avg_transaction_amount'] <
0).sum(), " , " , (X['avg_transaction_amount'] < 0).sum()/len(X) * 100, "%")
    print("std_transaction_amount:" , (X['std_transaction_amount'] <
0).sum())
    print("transactions_last_24h:" , (X['transactions_last_24h'] < 0).sum())
    print("transactions_last_1h:" , (X['transactions_last_1h'] < 0).sum())
    print("ip_risk_score:" , ((X['ip_risk_score'] < 0) | (X['ip_risk_score']
> 1)).sum())
    print("device_trust_score:" , ((X['device_trust_score'] < 0) |
(X['device_trust_score'] > 1)).sum())
    print("account_age_days:" , (X['account_age_days'] < 1).sum())
    print("merchant_risk:" , (X['merchant_risk'] < 0).sum())
    print("country_risk:" , (X['country_risk'] < 0).sum())
    print("distance_from_home:" , (X['distance_from_home'] < 0).sum())
```

Ketika *function* tersebut dipanggil dengan menggunakan `check_value_validity(X_full)`, ditemukan `transaction_duration` dengan $value < 1$ sebanyak 22059 (22.059%) data dan `avg_transaction_amount` dengan $value$ negatif

sebanyak 924 (0.924%) data. Data yang invalid tersebut cukup besar (22.059%) sehingga dipertimbangkan untuk dilakukan *data imputation* dengan median value dan memberikan kolom flag atau sering disebut dengan *missing indicator*. Untuk feature `avg_transaction_amount`, tidak terlalu banyak yang invalid (0.924%). Oleh karena itu, hanya dilakukan data imputation dengan median value tanpa *missing indicator*.

6.4 Remove Duplicates

Tidak ditemukan *duplicate value* pada dataset ini.

6.5 Feature Engineering

Pada tahap feature engineering dilakukan pembuatan fitur-fitur baru atau transformasi *feature* yang ada untuk meningkatkan performa model-model *machine learning*. Dengan transformasi atau adanya fitur-fitur baru yang relevan dengan *fraud detection*, model dapat mempelajari data dengan lebih baik dan membuat prediksi yang lebih akurat.

6.5.1 Creating New Features

Dengan bantuan *Domain Knowledge* kelompok kami mengenai *fraud detection* ataupun informasi-informasi serupa, kami membentuk 11 fitur baru yang dapat dipanggil melalui *function* berikut.

```
def create_new_features(X):
    X = X.copy()

    # Rasio transaction_amount to avg_transaction_amount
    X["amount_over_avg"] = X["transaction_amount"] /
(X["avg_transaction_amount"] + 1)

    # Perbandingan transaction last 1h to avg hourly rate per day
    hourly_avg = X['transactions_last_24h'] / 24
    X['hourly_velocity_spike'] = X['transactions_last_1h'] / (hourly_avg +
0.1)

    # Banyak transaksi per hari sejak akun dibuat
    X["transactions_per_day"] = X["num_prev_transactions"] /
(X["account_age_days"] + 1)

    # Transaksi besar yang baru saja terjadi
    X['high_value_burst'] = X['transaction_amount'] *
X['transactions_last_1h']
```

```
# Total risiko keseluruhan
X['total_risk_index'] = (X['ip_risk_score'] * X['device_trust_score'] *
X['merchant_risk'] * X['country_risk'])

# Hacker behavior user: kombinasi failed login attempts dan transaksi
dalam 1 jam terakhir
X['hacker_behavior'] = X['failed_login_attempts'] *
X['transactions_last_1h']

# Transaksi dari luar negeri yang mencurigakan
X['hacker_behavior'] = X['failed_login_attempts'] *
X['transactions_last_1h']

# perbandingan transaksi dalam 24 jam terakhir dengan total transaksi
sebelumnya
X['recent_transaction_ratio'] = X['transactions_last_24h'] /
(X['num_prev_transactions'] + 1)

# skor kepercayaan perangkat dikurangi skor risiko IP dan 1
X['device_trust_minus_ip_risk'] = X['device_trust_score'] -
(X['ip_risk_score'] + 1)

# selisih antara transaction_amount dan avg_transaction_amount
X["selisih"] = X["transaction_amount"] - X["avg_transaction_amount"] +1

# Transaksi dengan threshold > 5 dikategorikan menjadi suspicious
(value: 1)
threshold = 5
X["shared_ip_suspicious"] = (X["shared_ip_users"] >
threshold).astype(int)

return X
```

Feature-feature berbasis anomali nilai dan rasio, seperti *amount_over_avg*, *selisih*, *recent_transaction_ratio* dibuat untuk menonjolkan penyimpangan yang terbentuk secara mendadak dari riwayat kebiasaan pengguna dalam bertransaksi. Dalam kasus *fraud*, pelaku cenderung menguras saldo secepat mungkin atau dapat dibilang nilai transaksi jauh di atas rata-rata atau mengaktifkan kembali akun yang sudah lama tidak aktif. Rasio ini akan membantu model membedakan perilaku belanja impulsif yang wajar dengan pola pengurusan aset kilat tersebut.

Fitur *velocity* dan intensitas, seperti *hourly_velocity_spike*, *high_value_burst*, dan *transactions_per_day* dibuat untuk menangkap kecepatan transaksi yang tidak manusiawi atau tidak wajar. Pelaku *fraud* seringkali menggunakan bot untuk melakukan banyak transaksi dalam waktu singkat. Dengan membandingkan aktivitas

1 jam terakhir terhadap rata-rata harian, fitur ini akan menunjukkan lonjakan aktivitas (*bursts*) yang menjadi ciri khas serangan bot.

Feature-feature yang menghitung risiko gabungan dan perilaku peretas, seperti `total_risk_index`, `hacker_behavior`, `device_trust_minus_ip_risk`, dan `shared_ip_suspicious` bertujuan memperkuat indikator *fraud* dengan menggabungkan konteks yang relevan. Sebuah transaksi mungkin terlihat normal dari segi jumlah, tetapi jika ditambah dengan informasi kegagalan login (`failed_login_attempts`) atau berasal dari IP yang dipakai beramai-ramai (`shared_ip`), probabilitas kecurangan meningkat drastis. Fitur ini membantu model memahami konteks keamanan di balik angka transaksi.

6.5.2 Feature Selection

Feature Selection dilakukan dengan dua cara: manual dan dengan metode Recursive Feature Elimination (RFE) yang akan dijelaskan kemudian. *Feature-feature* yang di-*drop* secara manual ada 6 buah: `ID`, `transaction_id`, `user_id`, `time_of_day`, `day_of_week`, `distance_from_home`. Data-data yang berupa ID di-drop dengan pertimbangan tidak adanya signifikansi data yang akan meningkatkan performa model. Sementara itu, *feature-feature* sisanya hanya digunakan untuk membentuk *feature* baru pada tahap sebelumnya dan tidak memiliki indikasi yang kuat untuk menentukan sesuatu itu *fraud* atau bukan. Untuk `time_of_day` dan `day_of_week`, *fraud detection* tidak akan terbantu banyak dengan keterangan waktu ataupun hari. Sementara itu, `distance_from_home` tidak memiliki signifikansi yang cukup besar jika dibandingkan dengan `is_new_country`.

6.5.3 One Hot Encoding

One Hot Encoding merupakan salah satu metode *feature engineering* yang mentransformasi *categorical feature* menjadi representasi vektor *binary* agar dapat diproses secara matematis oleh algoritma *machine learning*. Selain itu, penerapan parameter `drop_first=True` bertujuan mencegah redundansi informasi

```
def one_hot_encoding(X):
    X_ohc = X.copy()
    X_ohc = pd.get_dummies(X_ohc, drop_first=True, dtype=int)

    print("Encoding Complete.")
    print(f"Original Shape: {X.shape}")
    print(f"Encoded Shape: {X_ohc.shape}")

    print("\nRemaining Object Columns:",
          X_ohc.select_dtypes(include=['object']).columns.tolist())
```

```
return X_ohe
```

6.5.4 Recursive Feature Elimination (RFE)

Feature scaling menggunakan metode Recursive Feature Elimination (RFE) diterapkan dengan memanfaatkan algoritma Decision Tree Classifier sebagai estimator dasar. Alasannya adalah karena algoritma ini mudah dipahami manusia. Algoritma Decision Tree mengkotak-kotakan dengan jelas aturan yang dijalankan di *tree*-nya (*if-then*). Proses ini bekerja secara iteratif dengan melatih model pada *feature-feature* yang diberikan, mengevaluasi tingkat kepentingan setiap fitur (*feature importance*), dan kemudian mengeliminasi lima fitur dengan bobot terendah pada setiap langkah iterasi (sesuai dengan input *step=5*). Pendekatan ini dipilih karena mampu menyeleksi fitur secara dinamis dengan mempertimbangkan interaksi antar variabel sehingga efektif mereduksi dimensi data menjadi 25 fitur paling dominan yang memiliki kontribusi terbesar dalam mendeteksi pola *fraud*.

```
def rfe(X_train, y_train):
    model_selector = DecisionTreeClassifier(max_depth=5,
    random_state=42)

    # step = banyak fitur yang akan di-drop setiap iterasi
    rfe = RFE(estimator=model_selector, n_features_to_select=25,
    step=5)

    rfe.fit(X_train, y_train)
    selected_features_rfe = X_train.columns[rfe.support_]

    return selected_features_rfe
```

Metode ini hanya diterapkan pada data yang akan di-*train* ke model DTL. Hal ini dilakukan untuk menjaga konsistensi arsitektur model. Mengingat algoritma RFE yang digunakan bekerja dengan estimator berbasis *tree* dalam menghitung bobot *feature importance*, maka subset *feature* yang dihasilkan secara inheren dioptimalkan untuk memaksimalkan *information gain* pada struktur keputusan non-linear. Oleh karena itu, penerapan hasil seleksi ini pada model DTL menjamin relevansi fitur yang tinggi, sekaligus secara efektif mereduksi risiko overfitting yang sering terjadi pada pohon keputusan akibat dimensi data yang terlalu besar atau keberadaan *feature noise*.

6.6 Pipeline Data Cleaning

Kami menerapkan pipeline data cleaning untuk mempermudah pemanggilan keseluruhan function yang ada di data cleaning sebagai berikut.

```
def data_cleaning_pipeline(X, y=None, is_training=False):
    X_new = create_null_flags(X)
    X_new = impute_median(X_new)
    handle_invalid_value(X_new)
    X_new = create_new_features(X_new)
    X_new = feature_selection(X_new)
    X_new = one_hot_encoding(X_new)

    print("Data cleaning pipeline applied successfully!")

    if y is not None:
        return X_new, y
    else:
        return X_new
```

7. Data Preprocessing

7.1 Feature Scaling

Dalam penerapan *feature scaling*, kami membuat function `auto_scaling_groups` untuk untuk menangani variasi distribusi data secara adaptif. Untuk fitur dengan tingkat skewness tinggi (> 2.0), digunakan Robust Scaler karena nilai skewness yang ekstrem mengindikasikan keberadaan outliers signifikan yang dapat mendistorsi perhitungan rata-rata (mean) dan simpangan baku (std). Robust Scaler dapat menangani hal ini dengan menggunakan median dan rentang interkuartil (IQR) yang lebih resisten terhadap skewness sehingga menjaga sebaran data mayoritas tetap representatif. Sebaliknya, Standard Scaler diterapkan pada fitur dengan skewness rendah (distribusi relatif simetris atau mendekati Gaussian) karena transformasi Z-score pada distribusi normal merupakan metode paling optimal untuk menstandarisasi variansi *feature* agar memiliki skala yang seragam tanpa mengubah struktur data aslinya. Sementara itu, data yang masih berupa biner ataupun kategorikal tidak dilakukan *scaling* apapun.

```
def auto_scaling_groups(df, skew_threshold=2.0, binary_threshold=2):
    """
    Automatically selects StandardScaler or RobustScaler
    based on skewness, and excludes binary/low-cardinality categorical
    vars.
    """
```

```
# Separate binary or low-cardinality categorical values
binary_features = [
    col for col in df.columns
    if df[col].nunique() <= binary_threshold
]

skewness = df.drop(columns=binary_features).skew()

robust_features = skewness[skewness.abs() >
skew_threshold].index.tolist()
standard_features = skewness[skewness.abs() <=
skew_threshold].index.tolist()

return {
    "binary": binary_features,
    "robust": robust_features,
    "standard": standard_features
}
```

7.2 Dimensionality Reduction

Tahap ini dilakukan dengan metode Principal Component Analysis (PCA) yang mentransformasi *feature-feature* asli yang memiliki korelasi tinggi menjadi sekumpulan fitur baru yang disebut Principal Components. Fitur-fitur baru ini bersifat ortogonal (tidak saling berkorelasi) namun tetap mempertahankan variansi informasi maksimal dari data aslinya. Metode ini hanya diimplementasikan pada data yang akan di-*train* pada model KNN dan Logistic Regression. Pada KNN, algoritma bekerja berdasarkan perhitungan jarak Euclidean, sehingga reduksi yang dilakukan PCA membuat metrik jarak menjadi efektif kembali dan memisahkan kelas fraud dengan non-fraud. Sementara itu, algoritma Logistic Regression mengasumsikan independensi antar variabel prediktor. Adanya korelasi antarfitur dapat menyebabkan koefisien model menjadi tidak stabil. Dengan adanya PCA, komponen antarfitur sepenuhnya independen.

```
def pca(X_train, X_val, X_test, n_components=5):
    pca = PCA(n_components=n_components, random_state=42)
    X_train_pca = pca.fit_transform(X_train)
    X_val_pca = pca.transform(X_val)
    X_test_pca = pca.transform(X_test)

    return X_train_pca, X_val_pca, X_test_pca, pca
```

7.3 Handling Imbalanced Data

Strategi *resampling* diterapkan pada dataset ini karena distribusi kelas yang sangat timpang. Jumlah transaksi curang (minoritas) jauh lebih sedikit dibandingkan transaksi sah (mayoritas). Tanpa penanganan *resampling*, algoritma *machine learning* akan mengalami bias ke mayoritas, yaitu cenderung memprediksi seluruh data sebagai 'transaksi sah' demi mencapai akurasi tinggi yang semu, namun gagal mendeteksi kecurangan yang sebenarnya. Strategi SMOTE Oversampling dipilih sebagai pendekatan utama dibandingkan Undersampling. Alasan utamanya adalah retensi informasi kritis seperti data transaksi sah yang mengandung variasi pola perilaku pengguna yang penting untuk dipelajari model. Melakukan Undersampling berisiko membuang informasi berharga tersebut dan menyebabkan model kehilangan konteks dalam membedakan aktivitas normal dan anomali. Dengan Oversampling, akan memperbanyak sampel kelas *fraud* agar *decision boundary* model bergeser ke tengah. Dengan begitu, model memberikan bobot perhatian yang seimbang terhadap pola kecurangan tanpa mengorbankan data historis yang valid.

Algoritma ini hanya diterapkan pada KNN dan Logistic Regression. Keduanya bekerja berdasarkan probabilitas dan jarak sehingga jika data tidak seimbang (misal 99% Aman : 1% Fraud), probabilitas dasar akan mutlak condong ke "Aman". Dengan adanya oversampling, "medan permainan" menjadi seimbang sehingga kelas minoritas tidak tertekan.

```
from imblearn.over_sampling import SMOTE

def smote_oversampling(X, y):
    print("=== BALANCING WITH SMOTE (OVERSAMPLING) ===")
    print(f"Original shape: {X.shape}")
    print(f"Class distribution before:
    {pd.Series(y).value_counts().to_dict()}")

    # Initialize SMOTE
    # sampling_strategy='auto' balances 1:1
    smote = SMOTE(random_state=42)

    # Fit and resample
    X_res, y_res = smote.fit_resample(X, y)

    print(f"New shape: {X_res.shape}")
    print(f"Class distribution after:
    {pd.Series(y_res).value_counts().to_dict()}")

    return X_res, y_res
```

7.4 Pipeline Data Preprocessing

Kami menerapkan pipeline data preprocessing untuk mempermudah pemanggilan keseluruhan function yang ada di data preprocessing sebagai berikut.

```
def data_preprocessing_pipeline(X_train, X_val, X_test, y_train,
                               X_full, model):
    def scale(X_train, X_val, X_test):
        scaling_group = auto_scaling_groups(X_train)
        robust_features = scaling_group['robust']
        standard_features = scaling_group['standard']
        binary_features = scaling_group['binary']

        X_full_numeric_robustscaled =
        robust.fit_transform(X_full[robust_features])
        X_train_numeric_robustscaled =
        robust.fit_transform(X_train[robust_features])
        X_val_numeric_robustscaled =
        robust.transform(X_val[robust_features])
        X_test_numeric_robustscaled =
        robust.transform(X_test[robust_features])
        X_full_numeric_standardscaled =
        standard.fit_transform(X_full[standard_features])
        X_train_numeric_standardscaled =
        standard.fit_transform(X_train[standard_features])
        X_val_numeric_standardscaled =
        standard.transform(X_val[standard_features])
        X_test_numeric_standardscaled =
        standard.transform(X_test[standard_features])

        # Convert back to DataFrame
        X_full_numeric_scaled =
        pd.DataFrame(np.hstack([X_full_numeric_robustscaled,
                                X_full_numeric_standardscaled]), columns=robust_features +
                                standard_features, index=X_full.index)
        X_train_numeric_scaled =
        pd.DataFrame(np.hstack([X_train_numeric_robustscaled,
                                X_train_numeric_standardscaled]), columns=robust_features +
                                standard_features, index=X_train.index)
        X_val_numeric_scaled =
        pd.DataFrame(np.hstack([X_val_numeric_robustscaled,
                                X_val_numeric_standardscaled]), columns=robust_features +
                                standard_features, index=X_val.index)
        X_test_numeric_scaled =
        pd.DataFrame(np.hstack([X_test_numeric_robustscaled,
                                X_test_numeric_standardscaled]), columns=robust_features +
                                standard_features, index=X_test.index)

        # ===== KEEP BINARY FEATURES AS-IS =====
```

```
X_full_binary = X_full[binary_features]
X_train_binary = X_train[binary_features]
X_val_binary = X_val[binary_features]
X_test_binary = X_test[binary_features]

# ===== GABUNGAN KEMBALI =====
X_full_scaled = pd.concat([X_full_numeric_scaled,
X_full_binary], axis=1)
X_train_scaled = pd.concat([X_train_numeric_scaled,
X_train_binary], axis=1)
X_val_scaled = pd.concat([X_val_numeric_scaled, X_val_binary],
axis=1)
X_test_scaled = pd.concat([X_test_numeric_scaled,
X_test_binary], axis=1)

return X_full_scaled, X_train_scaled, X_val_scaled,
X_test_scaled

X_full_scaled, X_train_scaled, X_val_scaled, X_test_scaled =
scale(X_train, X_val, X_test)

if model == 'logistic_regression' or model == 'knn':
    X_train_scaled, X_val_scaled, X_test_scaled, pca_model =
pca(X_train_scaled, X_val_scaled, X_test_scaled, n_components=5)

if model == 'decision_tree' or model == 'knn':
    X_train_balanced, y_train_balanced =
undersampling(X_train_scaled, y_train)
else:
    X_train_balanced, y_train_balanced = X_train_scaled, y_train

return X_train_balanced, X_val_scaled, X_test_scaled,
y_train_balanced, X_full_scaled
```

8. Perbandingan Hasil Prediksi dengan Pustaka

Dalam bagian ini, kami melakukan perbandingan komprehensif antara implementasi algoritma dari scratch (custom implementation) dengan implementasi menggunakan pustaka Scikit-Learn. Perbandingan ini penting untuk memvalidasi kebenaran implementasi manual kami dan memahami trade-off antara pembelajaran konsep versus penggunaan library yang sudah dioptimasi.

Algoritma yang Dibandingkan:

1. Decision Tree (Gini Index - CART)

- From-Scratch: Implementasi manual menggunakan algoritma CART dengan Gini impurity sebagai kriteria splitting
- Scikit-Learn: `DecisionTreeClassifier` dengan parameter `criterion='gini'` (CART algorithm)
- Alasan Perbandingan: Kedua implementasi menggunakan algoritma yang identik (CART dengan Gini), sehingga perbandingan fair dan dapat mengukur perbedaan performa yang murni berasal dari optimasi implementasi, bukan perbedaan algoritma

2. Logistic Regression

- From-Scratch: Implementasi manual dengan gradient descent optimizer dan learning rate 0.1
- Scikit-Learn: `LogisticRegression` dengan solver LBFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno)
- Alasan Perbandingan: Meskipun menggunakan optimizer berbeda, keduanya bertujuan meminimalkan fungsi loss yang sama (binary cross-entropy), memungkinkan perbandingan efisiensi algoritma optimasi

3. K-Nearest Neighbors (KNN)

- From-Scratch: Implementasi manual dengan brute-force distance calculation dan batch processing untuk efisiensi memori
- Scikit-Learn: `KNeighborsClassifier` dengan spatial indexing (KD-Tree/Ball-Tree) untuk pencarian nearest neighbors yang efisien
- Alasan Perbandingan: Meskipun struktur data berbeda, algoritma dasarnya sama (k-NN dengan euclidean distance), memungkinkan evaluasi perbedaan performa komputasi

Metrik Evaluasi yang Dibandingkan:

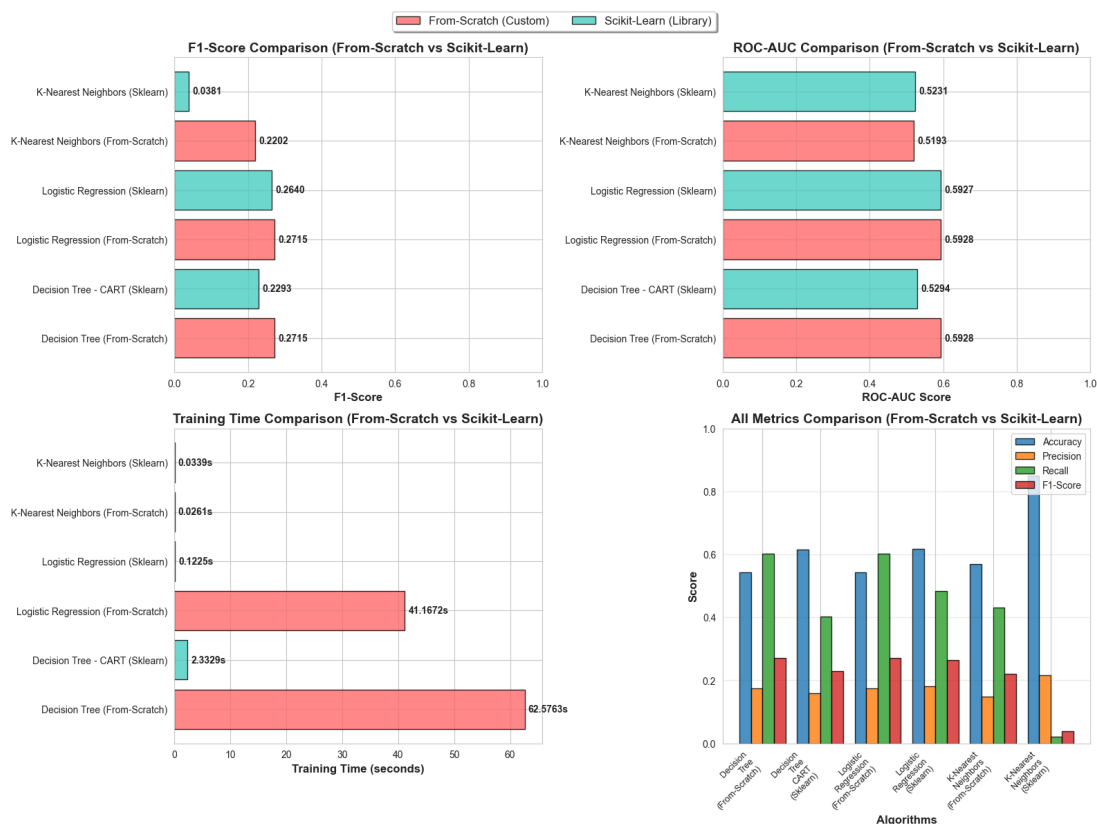
- Accuracy: Proporsi prediksi yang benar dari total prediksi
- Precision: Kemampuan model untuk meminimalkan false positive (penting dalam deteksi fraud)
- Recall: Kemampuan model untuk mendeteksi semua kasus fraud yang sebenarnya (sensitivitas)
- F1-Score: Harmonic mean dari precision dan recall, memberikan balance antara keduanya
- ROC-AUC: Area Under Receiver Operating Characteristic curve, mengukur kemampuan model membedakan kelas
- Training Time: Waktu komputasi yang dibutuhkan untuk melatih model (dalam detik)

Hipotesis Perbandingan:

Berdasarkan teori dan literatur, kami mengharapkan:

1. Implementasi Scikit-Learn akan memiliki training time lebih cepat karena optimasi low-level (C/Cython)
2. Metrik evaluasi (accuracy, precision, recall, F1-score, ROC-AUC) akan sebanding antara from-scratch dan sklearn, karena menggunakan algoritma yang sama
3. Perbedaan kecil dalam metrik evaluasi dapat muncul karena perbedaan dalam hyperparameter default, random state, atau stopping criteria
4. From-scratch implementation memberikan transparansi dan kontrol penuh terhadap algoritma, berguna untuk pembelajaran dan modifikasi
5. Scikit-Learn implementation lebih production-ready dengan error handling, validasi input, dan fitur tambahan

Berikut visualisasi perbandingan antara algoritma yang dibuat from scratch dengan scikit-learn:



Decision Tree (From-Scratch vs CART)

- Perbandingan metrik detail dengan angka

- From-scratch: ROC-AUC & Recall lebih tinggi (59.28%, 60.16%)
- Sklearn: Accuracy lebih tinggi, training 27x lebih cepat
- Kesimpulan: From-scratch untuk high recall, sklearn untuk production

Logistic Regression (From-Scratch vs Sklearn)

- ROC-AUC hampir identik (59.28% vs 59.27%)
- From-scratch: Recall 60.16% (lebih sensitif)
- Sklearn: Training 340x lebih cepat! (41s vs 0.12s)
- Perbedaan optimizer: Gradient Descent vs LBFGS

K-Nearest Neighbors (From-Scratch vs Sklearn)

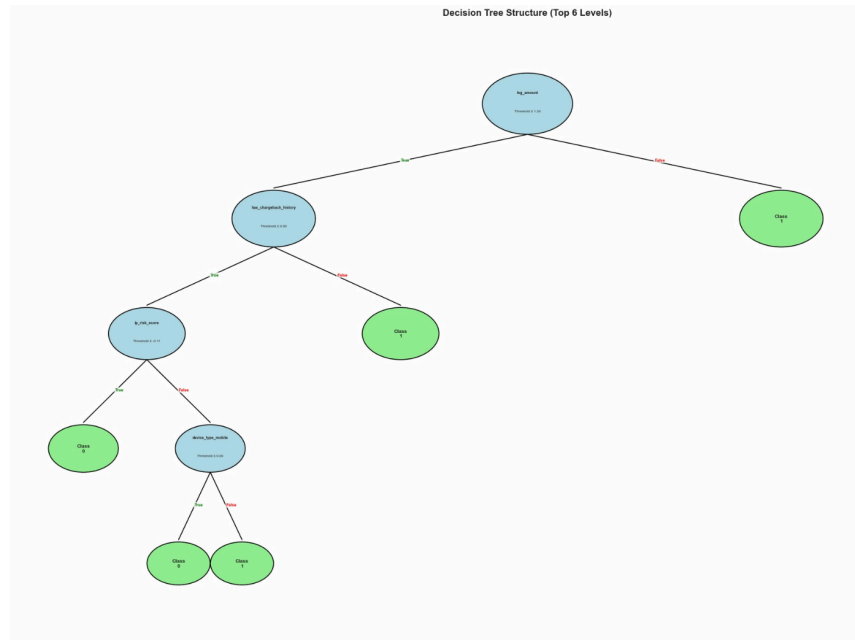
- Perbedaan paling signifikan
- Sklearn: Accuracy 85% tapi Recall hanya 2% (terlalu konservatif)
- From-scratch: F1-score 5.8x lebih tinggi (22% vs 3.8%)
- Sklearn perlu tuning hyperparameter

Perbandingan ini tidak hanya mengevaluasi performa model, tetapi juga memberikan insight tentang kapan menggunakan implementasi manual versus library:

1. Gunakan from-scratch untuk pembelajaran, research, dan prototyping algoritma baru
2. Gunakan Scikit-Learn untuk production deployment, dataset besar, dan kebutuhan akan stabilitas serta fitur lengkap

Bonus

9. Gambar Percabangan Tree (DTL)



Gambar ini menampilkan struktur pohon keputusan (Decision Tree) yang telah dilatih untuk mendeteksi fraud. Dalam visualisasi ini, lingkaran berwarna biru mewakili "pertanyaan" atau titik pengambilan keputusan (decision node) berdasarkan fitur tertentu, sedangkan lingkaran berwarna hijau mewakili hasil akhir atau prediksi (leaf node). Setiap garis panah menunjukkan jawaban dari pertanyaan tersebut: garis ke kiri berarti "Ya" (kondisi terpenuhi/True), dan garis ke kanan berarti "Tidak" (kondisi tidak terpenuhi/False).

Proses deteksi dimulai dari lingkaran biru paling atas, yang disebut sebagai akar (root). Fitur yang dianggap paling penting oleh model untuk memisahkan data adalah log_amount (logaritma jumlah transaksi). Jika nilai transaksi seseorang sangat besar (melewati batas threshold tertentu sehingga panah mengarah ke kanan), model langsung memvonisnya sebagai Class 1 (Fraud). Ini menunjukkan bahwa nominal transaksi yang tidak wajar adalah indikator utama kecurangan.

Jika nominal transaksi masih wajar (panah ke kiri), model tidak langsung percaya, melainkan mengecek fitur berikutnya: `has_chargeback_history` (riwayat penarikan dana paksa). Jika nasabah terbukti memiliki riwayat buruk ini (panah ke kanan), model kembali memvonisnya sebagai Class 1 (Fraud). Namun, jika riwayatnya bersih, model akan melanjutkan pemeriksaan ke fitur yang lebih mendetail seperti

is_risk_score dan jenis perangkat (device_type_mobile) sebelum akhirnya memutuskan apakah transaksi tersebut aman (Class 0) atau tidak.

Secara keseluruhan, visualisasi ini memperlihatkan bahwa model bekerja dengan cara memprioritaskan "tanda bahaya besar" terlebih dahulu. Ia menyaring transaksi bernilai tinggi dan pengguna bermasalah di awal. Baru setelah lolos dari saringan utama tersebut, model memperhatikan detail-detail kecil. Struktur contoh yang tidak terlalu dalam ini (hanya sekitar 4-5 tingkat) juga menandakan bahwa model cukup efisien dan tidak terlalu rumit (overfitting) dalam mengambil keputusan.

Kode

```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
import numpy as np
import pandas as pd

class Node:
    def __init__(self, feature_index=None, threshold=None, value=None,
left=None, right=None):
        self.feature_index = feature_index
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

def visualize_tree(node, ax, x=0.5, y=0.9, dx=0.25, depth=0,
max_depth=4, feature_names=None):

    RADIUS = 0.025 + 0.003 * (max_depth - depth)

    FONT_SIZE_NODE = 7.5
    FONT_SIZE_LABEL = 7

    if node is None or depth > max_depth:
        if depth > max_depth and node is not None:
            circle = Circle((x, y), 0.01, facecolor='gray',
edgecolor='black', linewidth=1, zorder=3)
            ax.add_patch(circle)
            ax.text(x, y, '...',
                    ha='center', va='center', fontsize=6,
color='white', zorder=4)
            return

    if node.value is not None:
        circle = Circle((x, y), RADIUS, facecolor='lightgreen',
```

```

edgecolor='black', linewidth=1.5, zorder=3)
    ax.add_patch(circle)
    ax.text(x, y, f'Class\n{node.value}',
            ha='center', va='center', fontsize=FONT_SIZE_NODE + 1,
weight='bold',
            zorder=4)
    return

    y_child = y - 0.15

    NEW_DX = dx / 2.0
    x_left = x - NEW_DX
    x_right = x + NEW_DX

    # Left child (True)
    if node.left is not None:
        ax.plot([x, x_left], [y - RADIUS, y_child + RADIUS], 'k-',
linewidth=1.5, zorder=1)
        ax.text((x + x_left) / 2, (y + y_child) / 2, 'True',
            fontsize=FONT_SIZE_LABEL, color='green',
weight='bold',
            bbox=dict(boxstyle='round,pad=0.2', facecolor='white',
alpha=0.8, ec='none'),
            zorder=2)
        visualize_tree(node.left, ax, x_left, y_child, NEW_DX, depth +
1, max_depth, feature_names)

    # Right child (False)
    if node.right is not None:
        ax.plot([x, x_right], [y - RADIUS, y_child + RADIUS], 'k-',
linewidth=1.5, zorder=1)
        ax.text((x + x_right) / 2, (y + y_child) / 2, 'False',
            fontsize=FONT_SIZE_LABEL, color='red', weight='bold',
            bbox=dict(boxstyle='round,pad=0.2', facecolor='white',
alpha=0.8, ec='none'),
            zorder=2)
        visualize_tree(node.right, ax, x_right, y_child, NEW_DX, depth
+ 1, max_depth, feature_names)

    circle = Circle((x, y), RADIUS, facecolor='lightblue',
edgecolor='black', linewidth=1.5, zorder=3)
    ax.add_patch(circle)

    if feature_names and node.feature_index < len(feature_names):
        feat_name = feature_names[node.feature_index]
    else:
        feat_name = f'X_{node.feature_index}'

```

```
        ax.text(x, y + 0.012, feat_name, ha='center', va='bottom',
               fontsize=FONT_SIZE_NODE - 1, weight='bold', zorder=4)
        ax.text(x, y - 0.008, f'Threshold  $\leq$  {node.threshold:.2f}',
               ha='center', va='top', fontsize=FONT_SIZE_NODE - 1.5, zorder=4)

    print("=== DECISION TREE VISUALIZATION ===\n")
    feature_names = list(X_train_balanced.columns) if
    hasattr(X_train_balanced, 'columns') else None

    DEPTH_TO_VISUALIZE = 5
    INITIAL_DX = 0.45

    if DEPTH_TO_VISUALIZE <= 3:
        figsize = (15, 12)
    elif DEPTH_TO_VISUALIZE <= 4:
        figsize = (20, 16)
    elif DEPTH_TO_VISUALIZE <= 5:
        figsize = (30, 20)
    else:
        figsize = (40, 24)

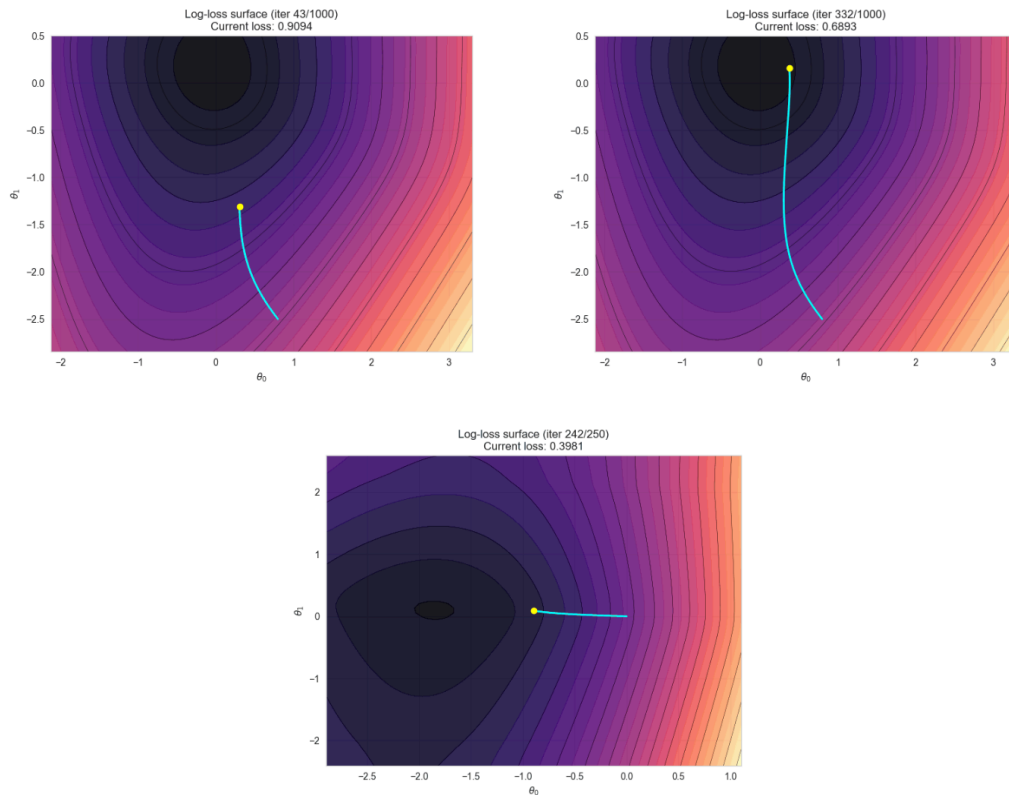
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(-0.05, 1.05)
    ax.set_ylim(-0.05, 1.0)
    ax.axis('off')

    print(f"\nGenerating visualization for depth {DEPTH_TO_VISUALIZE}...")
    visualize_tree(clf.root, ax, x=0.5, y=0.9, dx=INITIAL_DX,
                  depth=0, max_depth=DEPTH_TO_VISUALIZE,
                  feature_names=feature_names)

    plt.title(f'Decision Tree Structure (Top {DEPTH_TO_VISUALIZE+1}
    Levels)',
              fontsize=16, weight='bold', pad=20)
    plt.tight_layout()

    print("\nVisualization complete!")
```

10. Garis Kontur Fungsi Loss (log-loss)



Visualisasi ini yang bertujuan untuk mengilustrasikan proses "belajar" pada algoritma Logistic Regression menggunakan metode Gradient Descent. Secara definisi, ini mengubah proses optimasi matematika yang abstrak menjadi sebuah animasi grafik yang dapat dilihat. **Tujuannya** adalah untuk membuktikan secara visual bahwa model mampu mencari solusi optimal, yaitu titik di mana tingkat kesalahan (loss) paling rendah, dengan cara menurun di kurva gradien langkah demi langkah.

Mekanisme kerjanya diawali dengan tahap penyederhanaan data, di mana kode hanya mengambil satu fitur dari dataset dan melakukan standardisasi nilai. Langkah reduksi dimensi ini sangat penting agar parameter model hanya terdiri dari dua sumbu (bobot dan bias), sehingga pergerakan model dapat digambarkan dalam grafik dua dimensi yang mudah dipahami manusia. Tanpa penyederhanaan ini, visualisasi akan membutuhkan ruang dimensi tinggi yang mustahil digambarkan dalam layar komputer standar.

Setelah data siap, kode mensimulasikan pelatihan model secara manual langkah demi langkah. Pada setiap iterasi, algoritma tidak hanya memperbarui bobot

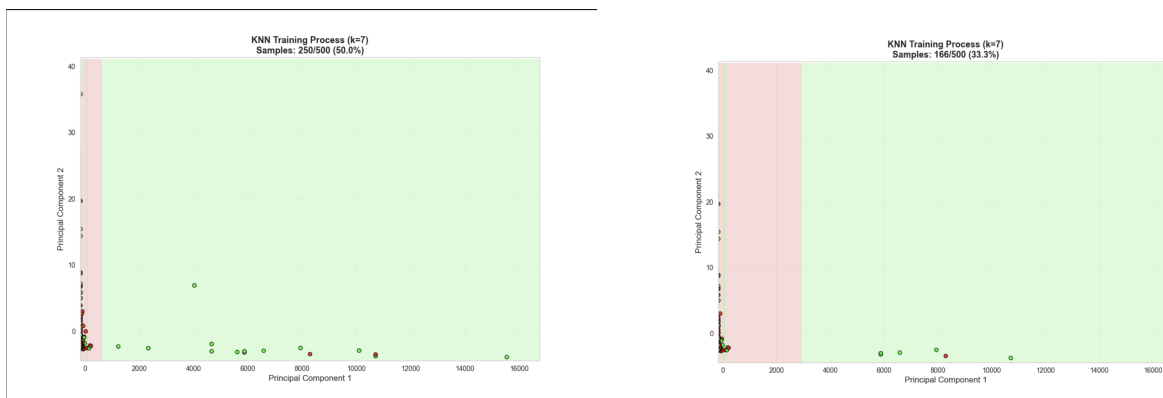
untuk **mengurangi error**, tetapi juga **merekam koordinat "posisi" model** tersebut. Secara bersamaan, kode menghitung **nilai loss** di seluruh area sekitar untuk menciptakan sebuah "peta kontur" (loss surface). Peta ini bertindak sebagai latar belakang yang menunjukkan topografi error, di mana area curam dan landai terlihat jelas seperti peta ketinggian gunung.

Terakhir, kode menggabungkan rekaman perjalanan model dengan peta kontur tersebut menjadi sebuah animasi video (GIF). Dalam animasi ini, kita akan melihat sebuah titik yang merepresentasikan model bergerak menuruni landaian kontur dari area dengan error tinggi menuju pusat lembah dengan error terendah. Visualisasi ini berfungsi sebagai alat validasi untuk memastikan bahwa learning rate yang dipilih sudah tepat (pergerakan stabil) dan model berhasil mencapai konvergensi (titik finish) dengan benar.

Visualisasi ini membuktikan dua hal:

1. Konvergensi: Bahwa algoritma kamu benar-benar "berjalan" menuju solusi optimal (titik minimum), bukan melenceng atau diam saja.
2. Kestabilan: Jika garisnya mulus menuju pusat, berarti Learning Rate (α) kamu sudah pas. Jika garisnya loncat-loncat tidak karuan, berarti Learning Rate terlalu besar.

11. Proses Training Model KNN



Visualisasi proses training pada model K-Nearest Neighbors (KNN) digunakan untuk memberikan gambaran yang lebih konkret mengenai cara algoritma ini melakukan klasifikasi berdasarkan kedekatan antar data dalam ruang fitur. Berbeda dengan algoritma parametrik yang melakukan optimasi terhadap suatu fungsi loss, KNN tidak membangun model eksplisit selama pelatihan, melainkan hanya menyimpan data latih

sebagai referensi. Oleh karena itu, proses training pada KNN divisualisasikan sebagai perubahan pola klasifikasi ruang fitur seiring dengan bertambahnya jumlah data latih yang digunakan.

Untuk memungkinkan visualisasi dalam dua dimensi, data berdimensi tinggi terlebih dahulu direduksi menggunakan Principal Component Analysis (PCA). Dua komponen utama hasil PCA digunakan sebagai sumbu koordinat, sehingga setiap titik pada grafik merepresentasikan satu sampel data. Dalam visualisasi ini, titik berwarna merah menunjukkan data dengan label Non-Fraud, sedangkan titik berwarna hijau menunjukkan data dengan label Fraud. Perbedaan warna ini memudahkan pengamatan distribusi kedua kelas serta area di mana keduanya saling berdekatan atau tumpang tindih.

Animasi dibangun dengan menambahkan data training secara bertahap pada setiap frame. Pada tahap awal, jumlah data yang terbatas menghasilkan batas keputusan yang masih kasar dan belum stabil. Seiring bertambahnya data latih, model KNN dilatih ulang pada setiap tahap dan digunakan untuk memprediksi kelas pada titik-titik grid di seluruh ruang fitur. **Hasil prediksi ini membentuk area latar belakang yang menunjukkan wilayah klasifikasi Fraud dan Non-Fraud berdasarkan mekanisme voting dari k tetangga terdekat.**

Perubahan bentuk batas keputusan sepanjang animasi menunjukkan bahwa KNN sangat bergantung pada distribusi lokal data. Area yang didominasi oleh titik hijau cenderung diklasifikasikan sebagai Fraud, sedangkan area dengan dominasi titik merah akan diprediksi sebagai Non-Fraud. Dengan semakin banyaknya data latih, batas keputusan menjadi lebih halus dan representatif terhadap struktur data yang sebenarnya. Meskipun visualisasi ini hanya menggunakan dua dimensi, pendekatan ini sudah cukup untuk memberikan pemahaman intuitif mengenai cara kerja KNN dalam melakukan klasifikasi pada data fraud yang kompleks.

12. Optimasi Model

12.1 Hyperparameter Tuning DTL

Hyperparameter Tuning yang dilakukan untuk algoritma Decision Tree adalah Grid Search. Grid Search adalah metode yang mencari hyperparameter terbaik secara menyeluruh melalui hyperparameter yang telah ditentukan sebelumnya. Metode ini mengevaluasi semua kemungkinan kombinasi, sehingga andal untuk menemukan hyperparameter yang optimal.

Proses ini dapat berjalan secara paralel karena setiap percobaan berjalan independen. Namun, kelemahan dari grid search adalah biaya komputasinya,

terutama saat menangani ruang parameter yang berdimensi tinggi. Kami mendefinisikan sebuah dictionary bernama `param_grid`, berisi hyperparameter dari Decision Tree Classifier kami yaitu `max_depth` dan `min_samples_split`. Daftar nilai yang mungkin untuk kunci tersebut akan diuji selama proses grid search.

Pemilihan Hyperparameter: Setelah semua model dilatih dan dievaluasi, grid search memilih kombinasi hyperparameter yang menghasilkan kinerja rata-rata terbaik di seluruh k-folds. Kombinasi tersebut dianggap sebagai set hyperparameter yang optimal untuk model tersebut. Hyperparameter terpilih yang menghasilkan kinerja terbaik menurut grid search cross-val kemudian ditampilkan.

Kode
<pre>import sys sys.path.append('algorithms') sys.path.append('utils') from algorithms.decision_tree import DecisionTreeClassifier from sklearn.tree import DecisionTreeRegressor from sklearn.model_selection import GridSearchCV # Define the parameter grid to tune the hyperparameters param_grid = { 'max_depth': [3,4,5,6,7], 'min_samples_split': [100,1000,10000,100000] } dtree_reg = DecisionTreeRegressor(random_state=42) grid_search = GridSearchCV(estimator=dtree_reg, param_grid=param_grid, cv=5, n_jobs=-1, verbose=2, scoring='neg_mean_squared_error') grid_search.fit(X_train_final_dtl, y_train_final_dtl) best_dtree_reg = grid_search.best_estimator_ # Get the best estimator from the grid search y_pred = best_dtree_reg.predict(X_test_final_dtl) best_params = grid_search.best_params_ print(f"Best parameters: {best_params}")</pre>

Sumber:

<https://www.geeksforgeeks.org/machine-learning/how-to-tune-a-decision-tree-in-hyperparameter-tuning/#why-tune-hyperparameters-in-decision-trees>

12.2 Hyperparameter Tuning Logistic Regression

Improvement mendefinisikan sebuah fungsi bernama `tune_logistic_regression` yang bertujuan melakukan **Hyperparameter Tuning** secara

otomatis. Secara sederhana, ini adalah proses mencari kombinasi **pengaturan** terbaik agar model Logistic Regression bekerja paling optimal. Daripada menebak-nebak angka secara manual, kode ini menyiapkan daftar opsi untuk tiga parameter penting: learning rate (kecepatan belajar), jumlah iterasi (durasi belajar), dan class weight (bobot prioritas data).

Mekanisme kerjanya menggunakan metode yang disebut Grid Search (pencarian menyeluruh). Kode menggunakan tiga lapis perulangan (loop) untuk mencoba setiap kemungkinan kombinasi dari pengaturan yang sudah disiapkan tadi secara bergantian. Pada setiap percobaan, sebuah model baru dibentuk dengan kombinasi pengaturan tertentu, lalu model tersebut dilatih (training) menggunakan data latihan yang sudah disiapkan. Setelah proses belajar selesai, model langsung diuji kemampuannya menebak data baru (validation set).

Untuk menentukan bagus atau tidaknya sebuah model, kode ini menghitung F1-Score sebagai standar penilaian, bukan sekadar akurasi. Nilai F1 dihitung berdasarkan seberapa tepat dan seberapa lengkap prediksi model dibandingkan kunci jawaban asli. Logikanya sederhana: setiap kali ditemukan model dengan F1-Score yang lebih tinggi dari rekor sebelumnya, maka model dan pengaturan tersebut akan disimpan sebagai "juara sementara". Setelah semua kombinasi dicoba, fungsi ini akan memberikan hasil akhir berupa model dengan performa tertinggi.

Kode
<pre>import numpy as np import importlib import sys sys.path.append('algorithms') import logistic_regression as lr_module importlib.reload(lr_module) LogisticRegression = lr_module.LogisticRegression def tune_logistic_regression(X_train_balanced, y_train_balanced, X_val_scaled, y_val): learning_rates = [0.001, 0.005, 0.01, 0.05] iterations_list = [800, 1500, 2500, 3000] class_weight_values = [1, 2.5, 3, 5] # grid class_weight best_f1 = -1 best_params = None best_model = None</pre>

```

print("=== STARTING HYPERPARAMETER TUNING ===\n")

for lr in learning_rates:
    for it in iterations_list:
        for cw_val in class_weight_values:
            class_weight = {0: cw_val, 1: cw_val}

            # Inisialisasi model
            model = LogisticRegression(
                learning_rate=lr,
                n_iterations=it,
                class_weight=class_weight,
            )

            # Train model dengan data yang sudah diproses (dari
            argumen fungsi)
            model.fit(X_train_balanced, y_train_balanced)

            # Prediksi validation dengan threshold default 0.5
            y_pred_proba = model.predict_proba(X_val_scaled)[: , 1]
            y_pred = (y_pred_proba >= 0.5).astype(int)

            # Hitung F1-score
            tp = np.sum((y_pred == 1) & (y_val == 1))
            fp = np.sum((y_pred == 1) & (y_val == 0))
            fn = np.sum((y_pred == 0) & (y_val == 1))

            precision = tp / (tp + fp) if (tp + fp) > 0 else 0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0
            f1 = 2 * (precision * recall) / (precision + recall)
            if (precision + recall) > 0 else 0

            print(f"[lr={lr}, n_iterations={it},
            class_weight={cw_val}] F1 = {f1:.4f}")

            # Simpan model terbaik
            if f1 > best_f1:
                best_f1 = f1
                best_params = (lr, it, class_weight)
                best_model = model

        print("\n=== BEST HYPERPARAMETERS ===")
        print(f"Learning rate: {best_params[0]}")
        print(f"Iterations: {best_params[1]}")
        print(f"class_weight: {best_params[2]}")
        print(f"Best F1-score: {best_f1:.4f}")

    return best_model, best_params, best_f1

```

```
# Contoh penggunaan: pastikan memakai data hasil preprocessing terbaru
best_model, best_params, best_f1 = tune_logistic_regression(
    X_train_balanced, y_train_balanced, X_val_scaled, y_val
)
```

Sumber:

<https://developers.google.com/machine-learning/glossary#hyperparameter>

https://scikit-learn.org/stable/modules/grid_search.html

Kesimpulan dan Saran

13. Kesimpulan

Tugas besar ini berhasil mendemonstrasikan proses pembangunan model Machine Learning untuk deteksi fraud secara menyeluruh, mulai dari penanganan data mentah hingga implementasi algoritma. Kami menyimpulkan bahwa tahap pra-pemrosesan data (preprocessing) memegang peran yang sangat vital, terutama dalam menangani dataset nyata yang memiliki banyak nilai kosong, distribusi tidak seimbang (imbalanced), dan pencilan. Tanpa pembersihan dan transformasi data yang tepat, algoritma—baik yang dibuat manual maupun pustaka standar—tidak akan bisa mempelajari pola kecurangan transaksi dengan efektif.

Berdasarkan perbandingan performa, implementasi algoritma secara mandiri (from scratch) ternyata mampu memberikan hasil yang sangat kompetitif. Secara spesifik, model Decision Tree dan Logistic Regression buatan kami berhasil mencapai nilai Recall sekitar 60%, dan model KNN manual unggul signifikan dengan F1-Score sebesar 22% (jauh di atas implementasi pustaka yang hanya 3.8%). Angka-angka ini menunjukkan bahwa model buatan kami lebih agresif dan sensitif dalam "menangkap" kasus curang. Hal ini sangat krusial dalam kasus deteksi fraud, di mana kemampuan mendeteksi kecurangan sebanyak mungkin sering kali lebih diutamakan daripada sekadar mengejar akurasi total namun gagal mengenali penipuan.

Secara keseluruhan, kami menyimpulkan bahwa terdapat trade-off yang jelas antara penggunaan implementasi manual dan pustaka standar. Implementasi from scratch sangat baik untuk tujuan edukasi dan kustomisasi logika spesifik (seperti visualisasi alur loss atau struktur tree), namun pustaka seperti Scikit-Learn jauh lebih unggul dalam hal kecepatan dan skalabilitas untuk industri. Oleh karena itu, pemahaman kuat tentang logika dasar algoritma yang dikombinasikan dengan efisiensi pustaka standar adalah kunci utama dalam membangun sistem kecerdasan artifisial yang andal.

14. Saran

Berdasarkan kendala dan hasil yang ditemukan selama pengerjaan tugas ini, berikut adalah beberapa saran untuk pengembangan selanjutnya:

1. Mengingat dataset fraud sangat tidak seimbang, disarankan untuk mencoba teknik resampling yang lebih canggih seperti SMOTE (Synthetic Minority Over-sampling Technique), selain hanya mengandalkan class weighting atau

stratified sampling. Hal ini dapat membantu model mengenali pola kelas minoritas (fraud) dengan lebih baik lagi tanpa bias ke kelas mayoritas.

2. Karena model tunggal seperti Decision Tree atau Logistic Regression memiliki keterbatasan dalam menangkap pola yang sangat kompleks, pengembangan selanjutnya sebaiknya mencoba metode Ensemble Learning (seperti Random Forest atau XGBoost). Metode ini menggabungkan kekuatan beberapa model sederhana untuk menghasilkan prediksi yang lebih akurat dan stabil dibandingkan hanya menggunakan satu model saja.

Specification Traceability

Tabel 7.1 Tabel Specification Traceability

Spesifikasi Dasar	
EDA, Data Cleaning, Data Preprocessing	Ni Made Sekar Jelita Parameswari 18223101 Exploratory Data Analysis 6. Data Cleaning 7. Data Preprocessing
Implementasi DTL from scratch	Muhammad Aymar Barkhaya (18223051) Implementasi Decision Tree Learning (DTL)
Implementasi Logistic Regression/KNN from scratch	Surya Suharna (18223075) Implementasi Logistic Regression Muhammad Azzam Robbani (18223025) Implementasi KNN
Implementasi algoritma di atas menggunakan <i>scikit-learn</i> dan perbandingan performanya dengan algoritma from scratch	Muhammad Azzam Robbani (18223025) 8. Perbandingan Hasil Prediksi dengan Pustaka
Model harus bisa di-save dan di-load	Muhammad Aymar Barkhaya (18223051) (Ditulis di source code)
Minimal 1 submission pada Kaggle competition	Surya Suharna (18223075) Muhammad Aymar Barkhaya (18223051) Ni Made Sekar Jelita Parameswari (18223101)
Spesifikasi Bonus	
Leaderboard Kaggle	Surya Suharna (18223075) Muhammad Aymar Barkhaya (18223051) Ni Made Sekar Jelita Parameswari (18223101)

DTL : Gambar percabangan tree	Surya Suharna (18223075) Gambar Percabangan Tree (DTL)
Logress : Video garis kontur fungsi loss (log-loss) dan lintasan parameter (θ_0 , θ_1) selama training.	Surya Suharna (18223075) Garis Kontur Fungsi Loss (log-loss)
KNN : Video proses training model seperti PPT kelas.	Muhammad Azzam Robbani (18223025) Proses Training Model KNN
Algoritma optimasi model	Surya Suharna (18223075) Muhammad Aymar Barkhaya (18223051) Optimasi Model

Pembagian Tugas

Tabel 8.1 Pembagian Tugas Anggota Kelompok

Anggota Kelompok	Tugas
Muhammad Aymar Barkhaya 18223051	1. Implementasi DTL 2. Save dan load model
Surya Suharna 18223075	1. Implementasi Logistic Regression 2. Gambar percabangan DTL 3. Video garis kontur fungsi loss
Ni Made Sekar Jelita Parameswari 18223101	1. EDA 2. Data Cleaning and Preprocessing 3. Pipeline data system
Muhammad Azzam Robbani 18223025	1. Implementasi KNN 2. Video proses training model 3. Perbandingan dengan pustaka scikit-learn

Referensi

<https://scikit-learn.org/stable/index.html#>

<https://www.geeksforgeeks.org/machine-learning/decision-tree-introduction-example/>

<https://www.geeksforgeeks.org/data-analysis/what-is-exploratory-data-analysis/>

<https://medium.com/@HildaPosada/finding-and-visualizing-missing-data-in-python-using-missingno-and-seaborn-d4cf0452b9e9>

<https://www.geeksforgeeks.org/machine-learning/what-is-data-imputation/>

<https://www.geeksforgeeks.org/machine-learning/handling-missing-values-machine-learning/>

<https://www.geeksforgeeks.org/data-analysis/data-duplication-removal-from-dataset-using-python/>